

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR

UNIVERSITÉ DE DOUALA

FACULTE DES SCIENCES

ECOLE SUPERIEURE DES SCIENCES DE LA
FINANCE, DE L'ASSURANCE ET DES RISQUES



REPUBLIC OF CAMEROON

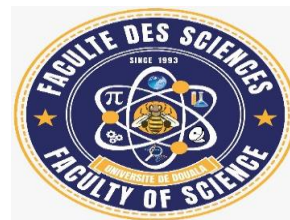
Peace-Work-Fatherland

MINISTRY OF HIGHER EDUCATION

DOUALA UNIVERSITY

FACULTY OF SCIENCES

HIGHER SCHOOL OF FINANCE,
INSURANCES AND RISK SCIENCES



Mémoire de fin d'études en vue de l'obtention du diplôme de Master en Mathématiques et Informatique Appliquées

Spécialité : Statistiques, Big Data et Intelligence Artificielle

Thème :

Étude et mise en œuvre des bases de données vectorielles
dans l'écosystème Oracle : intégration au sein d'une
architecture Data Lake, cas d'usage et analyse des
performances

Présenté par :

M. Kounga Tchomnou Ryan Yvan

Matricule :

Encadreur académique :

Dr. GABRIEL MOPOLLO MOKE

Docteur en Informatique

Année 2025

Résumé

L'évolution massif des données non structurées, telles que les textes, les images, les vidéos et les contenus multimédias, constitue aujourd'hui un enjeu majeur pour les systèmes d'information modernes. Ces données, qui représentent la majorité des volumes de données produits à l'échelle mondiale, ne peuvent plus être efficacement exploitées par les bases de données relationnelles traditionnelles. Dans ce contexte, les bases de données vectorielles émergent comme une solution adaptée, permettant le stockage, l'indexation et la recherche de similarité à partir de représentations vectorielles issues de modèles d'intelligence artificielle.

Ce mémoire s'inscrit dans cette dynamique et a pour objectif d'étudier les bases de données vectorielles et leur intégration dans l'écosystème Oracle, à travers la version Oracle Database 23ai. Le travail débute par une étude théorique des concepts fondamentaux liés aux données vectorielles, aux algorithmes de recherche de similarité et aux architectures associées. Il se poursuit par un état de l'art comparatif des principales solutions existantes, incluant Oracle AI Vector, pgvector, Weaviate et FAISS.

Une partie importante du travail est consacrée à l'installation, la configuration et la prise en main d'Oracle Database 23ai, suivie d'une expérimentation pratique basée sur un cas d'usage en traitement automatique du langage naturel (NLP). Les vecteurs sont générés à l'aide de modèles de type Sentence Transformers, stockés dans Oracle et interrogés via des requêtes SQL de similarité. Enfin, le mémoire explore l'intégration d'Oracle 23ai dans une architecture Big Data / Data Lake, mettant en évidence les apports, les performances et les limites de cette approche.

Les résultats obtenus démontrent la pertinence d'Oracle Database 23ai comme moteur vectoriel intégré, capable de répondre à des cas d'usage modernes en intelligence artificielle, tout en s'appuyant sur la robustesse d'un SGBD relationnel mature.

Abstract

The massive growth of unstructured data, such as text, images, videos, and multimedia content, has become a major challenge for modern information systems. As these data now represent the majority of global data volumes, traditional relational databases are no longer sufficient to efficiently store and exploit them. In this context, vector databases have emerged as a key solution, enabling the storage, indexing, and similarity search of high-dimensional vector representations generated by artificial intelligence models.

This thesis focuses on vector databases and their integration within the Oracle ecosystem, particularly through Oracle Database 23ai. The study begins with a theoretical overview of the fundamental concepts of vector data, similarity search algorithms, and vector database architectures. It then presents a comparative state of the art of major existing solutions, including Oracle AI Vector, pgvector, Weaviate, and FAISS.

A significant part of the work is dedicated to the installation, configuration, and hands-on experimentation with Oracle Database 23ai. A practical use case based on Natural Language Processing (NLP) is implemented, where textual data are transformed into vector embeddings using Sentence Transformer models, stored in Oracle, and queried through SQL-based similarity searches. Finally, the thesis explores the integration of Oracle 23ai into a Big Data / Data Lake architecture, highlighting its benefits, performance, and limitations.

The results demonstrate that Oracle Database 23ai provides a robust and integrated vector database solution, well suited for modern AI-driven applications while leveraging the reliability of a mature relational database system.

Liste des figures

Liste des acronymes

Acronyms	Signification
AI	Artificial Intelligence
ANN	Approximate Nearest Neighbors
<i>TF-IDF</i>	Terms Frequency Inverse Document Frequency
HNSW	Hierarchical Navigable Small World graphs
IVF	Inverted File Index
LSH	Locality-Sensitive Hashing
BD	Base de Données
CDB	Container Database
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
ETL	Extract, Transform, Load
HNSW	Hierarchical Navigable Small World
PQ	Product Quantization
API	Application Programming Interface
IA	Intelligence Artificielle
LLM	Large Language Model
NLP	Natural Language Processing
PCA	Principal Component Analysis
PDB	Pluggable Database
RAG	Retrieval Augmented Generation
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language

Table des matières

1	Introduction Générale	1
2	Les concepts fondamentaux des bases de données vectorielles	3
2.1	Les bases de données vectorielles	3
2.1.1	Définition	3
2.1.2	Représentation des vecteurs	3
2.1.3	Recherche vectorielle : recherche de similarité et métriques	4
2.1.4	Méthodes : brute force vs approximate	5
2.2	Architectures des systèmes vectoriels	5
2.3	Domaine d'utilisation des bases de données vectorielles	7
3	État de l'art sur les moteurs de bases de données vectorielles	8
3.1	Critères d'évaluation	8
3.2	Comparaison des différentes Base de données Vectorielles	9
3.2.1	Oracle AI Vector	9
3.2.2	Pgvector	10
3.2.3	Weaviate	12
3.2.4	FAISS (Facebook AI Similarity Search)	13
3.3	Tableau comparatif des moteurs de BD vectoriels	14
4	Oracle Data base 23ai et la gestion des bases de données vectorielles	16
4.1	Objectifs et positionnement d'Oracle Database 23ai	16
4.2	Composants et mécanismes vectoriels d'Oracle Database 23ai	17
4.3	Architecture distribuée, scalabilité et résilience	18
5	Installation d'Oracle 23AI	19
5.1	Contexte et enjeux	19
5.2	Objectifs de cette première phase	19
5.3	Prérequis matériels et logiciels	20
5.3.1	Environnement matériel utilisé	20
5.3.2	Choix du system d'exploitations	20
5.3.3	Téléchargement	21
5.3.4	Validation des prérequis	21
5.3.5	Bilan de la phase préparatoire	21
5.4	Choix de la version : Oracle Database 23ai	21
5.4.1	Téléchargement de l'archive Oracle	22
5.4.2	Préparation avant exécution	22
5.5	Installation et post-installation	23
5.5.1	Démarrage de l'installation	23
5.5.2	Structure des dossiers et chemins d'installation	23

5.5.3	Services Oracle installés	24
5.5.4	Premier test de connexion avec SQL*Plus.....	24
5.5.5	Connexion à la base PDB.....	25
5.5.6	Résultat de l'installation	26
6	MANIPULATION ET PRISE EN MAIN D'ORACLE 23ai.....	26
6.1	Manipulations de base : création d'utilisateurs, tables, insertion de données et requêtes SQL	26
6.1.1	Création d'un utilisateur Oracle personnalisé	26
6.1.2	Connexion avec l'utilisateur créé.....	27
6.1.3	Création d'une table simple	27
6.1.4	Insertion de données.....	27
6.1.5	Requêtes SQL de lecture	28
6.1.6	Mise à jour et suppression de données	28
6.1.7	Synthèse	29
6.2	Prise en main des bases de données vectorielles avec Oracle 23ai.....	29
6.2.1	Contexte et motivation	29
6.2.2	Recherche vectorielle intégrée	29
6.2.3	Création d'une table plus avancée : vecteurs NLP	30
6.2.4	Intégration avec des modèles IA et NLP	30
6.2.5	Insertion des données vectorielles en Python.....	30
6.2.6	Requêtes SQL sur la table (en Python)	31
6.2.7	Résumé.....	32
6.2.8	Performances et observations	33
6.2.9	Synthèse	33
6.3	Conclusion de la phase d'installation et de prise en main	34
7	Intégration d'oracle 23ai et les bases de données vectorielles dans un environnement Bigdata/data lake	35
7.1	Introduction	35
7.2	Application de référence et contexte expérimental.....	35
7.3	Architecture générale du Data Lake	36
7.3.1	Sources de données	36
7.3.2	Intégration via MySQL et Apache Hop	37
7.3.3	Passerelle ODBC entre Oracle et MySQL	38
7.3.4	Stockage centralisé dans Oracle 23AI.....	39
7.4	Oracle 23AI comme moteur vectoriel dans le Data Lake.....	39
7.4.1	Génération des vecteurs	39
7.4.2	Stockage des vecteurs dans Oracle.....	42
7.5	Analyses avancées dans le Data Lake	43
7.5.1	PCA pour visualiser la distribution des vecteurs	43

7.5.2	Regroupement de patients par similarité vectorielle avec DBSCAN	46
7.6	Analyse et évaluation expérimentale des performances de la recherche de similarité vectorielle dans Oracle 23 AI.....	49
7.6.1	Indicateurs de performance mesurée.....	51
7.6.2	Protocole expérimental.....	52
7.6.3	Objectif final de l'évaluation	52
7.6.4	Résultats.....	52
7.6.5	Tableau de synthèse des performances	53
7.6.6	Graphique d'analyse des performances.....	53
7.6.7	Interprétation générale	54
7.7	Apport de l'intégration d'Oracle 23AI dans un Data Lake	54
7.8	Conclusion.....	55
8	Conclusion générale et perspectives	56
9	Bibliographie	58

1 Introduction Générale

D'après IDC, la croissance des données non structurées constitue un enjeu majeur pour les entreprises, celles-ci représentant environ 80 % de la masse mondiale de données prévue à l'horizon 2025. Cette explosion des données, couplée à l'évolution de l'intelligence artificielle, a profondément transformé les besoins en matière de stockage et de recherche d'information. Les approches traditionnelles basées sur les bases de données relationnelles montrent leurs limites face à des usages nécessitant une compréhension sémantique des données. C'est dans ce contexte que les bases de données vectorielles se sont imposées comme une solution clé pour la recherche de similarité et l'exploitation avancée des données non structurées. (Rubrik 2025)

Le choix de cette étude s'inscrit dans la volonté d'explorer ces nouvelles approches de gestion de données, en particulier au sein de l'écosystème Oracle. L'introduction des fonctionnalités vectorielles dans Oracle Database 23ai représente une évolution majeure, visant à rapprocher les capacités de l'intelligence artificielle des systèmes de gestion de bases de données relationnelles existants. Ce travail a ainsi été motivé par l'intérêt de comprendre le fonctionnement de ces mécanismes, d'en évaluer les performances et d'analyser leur pertinence dans des cas d'usage concrets, notamment dans le domaine médical.

L'objectif principal de cette étude est d'analyser les bases de données vectorielles et leur intégration dans Oracle Database 23ai. Plus précisément, il s'agit d'étudier les principes de la recherche de similarité vectorielle, de comparer les principales solutions existantes, de mettre en œuvre un prototype basé sur des données textuelles et des images, et d'évaluer les performances obtenues en conditions réelles. Les attentes de ce travail portent également sur la validation de la capacité d'Oracle à supporter des cas d'usage orientés intelligence artificielle et à s'intégrer dans une architecture de type Data Lake.

Dans le cadre de cette étude, plusieurs missions ont été menées, incluant l'installation et la configuration d'Oracle Database 23ai, la génération et la vectorisation de données textuelles et visuelles, la mise en place de requêtes de similarité vectorielle, l'analyse des performances du moteur Oracle, ainsi que l'étude de son intégration dans une architecture Big Data.

La problématique centrale de ce rapport peut ainsi être formulée de la manière suivante : comment exploiter les capacités vectorielles d'Oracle Database 23ai pour réaliser des recherches de similarité performantes et pertinentes, tout en garantissant leur intégration dans une architecture Data Lake et leur adéquation à des cas d'usage réels basés sur l'intelligence artificielle ?

Ce mémoire s'articule autour de plusieurs chapitres. Il débute par une présentation des concepts fondamentaux des bases de données vectorielles et des principes de la recherche par similarité. Il se poursuit par un état de

l'art des principaux moteurs de bases de données vectorielles, avec une analyse comparative des solutions existantes, notamment Oracle AI Vector, pgvector, Weaviate et FAISS. Le document aborde ensuite l'architecture, l'installation et la prise en main d'Oracle Database 23ai, ainsi que la mise en œuvre pratique de la recherche vectorielle intégrée. Enfin, le mémoire traite de l'intégration d'Oracle 23ai dans un environnement Big Data et Data Lake, et présente des analyses avancées, incluant l'évaluation des performances et des cas d'usage appliqués au domaine médical, avant de conclure sur les résultats obtenus et les perspectives futures.

2 Les concepts fondamentaux des bases de données vectorielles

2.1 Les bases de données vectorielles

2.1.1 Définition

Une base de données vectorielle est un système de gestion de données conçu pour stocker, indexer et interroger des données vectorielles de grande dimension. Les informations y sont représentées sous forme de vecteurs numériques, permettant le regroupement et la recherche d'éléments en fonction de leur similarité sémantique. Cette architecture favorise des temps de réponse faibles, ce qui la rend particulièrement adaptée aux applications d'intelligence artificielle. (Matthew et Jim s.d.)

Contrairement aux bases relationnelles traditionnelles, qui manipulent des données tabulaires, les bases vectorielles travaillent avec des vecteurs, c'est-à-dire des représentations mathématiques en haute dimension produites par des modèles d'apprentissage automatique. Ces vecteurs capturent la sémantique ou les caractéristiques essentielles d'un objet (le sens d'une phrase, les traits visuels d'une image ou la signature audio d'un signal).

2.1.2 Représentation des vecteurs

Deux grandes formes de représentations coexistent :

- **Vecteurs denses** : ils sont généralement produits par des modèles de type deep Learning (par exemple, les *transformer* ou les réseaux convolutifs). Chaque dimension contient une valeur réelle, et l'ensemble des dimensions encode des informations compactes sur le contenu. Ces représentations sont largement utilisées pour la recherche sémantique, la reconnaissance d'images ou encore la recommandation.

Exemple : Vecteur de dimension 6

[0.12, -0.45, 0.88, 0.03, -0.27, 0.61]

- **Vecteurs clairsemés (sparse)** : ils proviennent souvent de méthodes classiques comme le *bag-of-Word* ou la pondération *TF-IDF*. Les vecteurs sont de grande dimension, mais contiennent majoritairement des zéros. Bien que plus simples, ils sont moins efficaces pour capturer la sémantique profonde des données.

Exemple : Vecteur de dimension 10

[0, 0, 3, 0, 0, 0, 1, 0, 0, 0]

2.1.3 Recherche vectorielle : recherche de similarité et métriques

La recherche vectorielle consiste à retrouver, pour un vecteur donné (représentant une requête), les vecteurs les plus proches dans l'espace multidimensionnel. Cette proximité est mesurée par une **fonction de distance** ou **métrique de similarité**. Les principales métriques utilisées sont :

1. Distance Euclidienne (L2 Norme)

La **distance euclidienne** est l'une des métriques les plus utilisées.

Pour deux vecteurs $x = (x_1, x_2, \dots, x_n)$ et $y = (y_1, y_2, \dots, y_n)$:

$$d_{\{euclidienne\}(x,y)} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. Distance de Manhattan (L1 Norme)

Elle calcule la somme des différences absolues entre les coordonnées :

$$d_{\{euclidienne\}(x,y)} = \sum_{i=1}^n |x_i - y_i|$$

3. Distance Cosinus (Cosine Similarity)

Très utilisée en NLP et pour la recherche sémantique, car elle mesure l'angle entre deux vecteurs (indépendamment de leur norme) :

$$sim_{\{cos\}}(x, y) = \frac{x \cdot y}{||x|| \cdot ||y||} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \cdot \sqrt{\sum_{i=1}^n y_i^2}}$$

La distance cosinus peut être dérivée comme :

$$d_{\{cos\}}(x, y) = 1 - sim_{\{cos\}}(x, y)$$

Ces métriques déterminent la qualité et la pertinence des résultats lors des recherches de similarité

(Christopher D. , Prabhakar et Hinrich 2009)

2.1.4 Méthodes : brute force vs approximate

Deux grandes approches se distinguent :

- **Brute Force (exhaustive search)** : consiste à comparer chaque vecteur avec l'ensemble de la base. Bien que simple et garantissant des résultats exacts, cette méthode devient coûteuse en temps de calcul dès que la base atteint des millions de vecteurs.
- **Approximate Nearest Neighbor (ANN)** : repose sur des algorithmes d'indexation avancés qui accélèrent la recherche en sacrifiant légèrement la précision. Parmi les techniques les plus utilisées, on retrouve :
 - **HNSW** efficace pour de grandes bases.
 - **IVF** souvent couplé à une quantification (PQ, OPQ).
 - **LSH** basé sur des fonctions de hachage adaptées aux distances.

Ces méthodes permettent d'obtenir des recherches en temps quasi réel, même sur des bases massives. (Jeff , Matthijs and Herve 2017) (Jingdong , Ting et Jingkuan APRIL 2017)

2.2 Architectures des systèmes vectoriels

Les systèmes de bases de données vectorielles reposent sur une architecture spécifique permettant de traiter efficacement des données non structurées à grande échelle. Cette architecture combine des modèles d'intelligence artificielle pour la génération de représentations vectorielles, des mécanismes de stockage adaptés aux vecteurs de grande dimension, ainsi que des algorithmes optimisés pour la recherche de similarité. L'objectif principal est de garantir des temps de réponse faibles tout en assurant une bonne précision des résultats, même sur des volumes de données importants. La Figure 1 illustre les principales composantes de cette architecture et leurs interactions

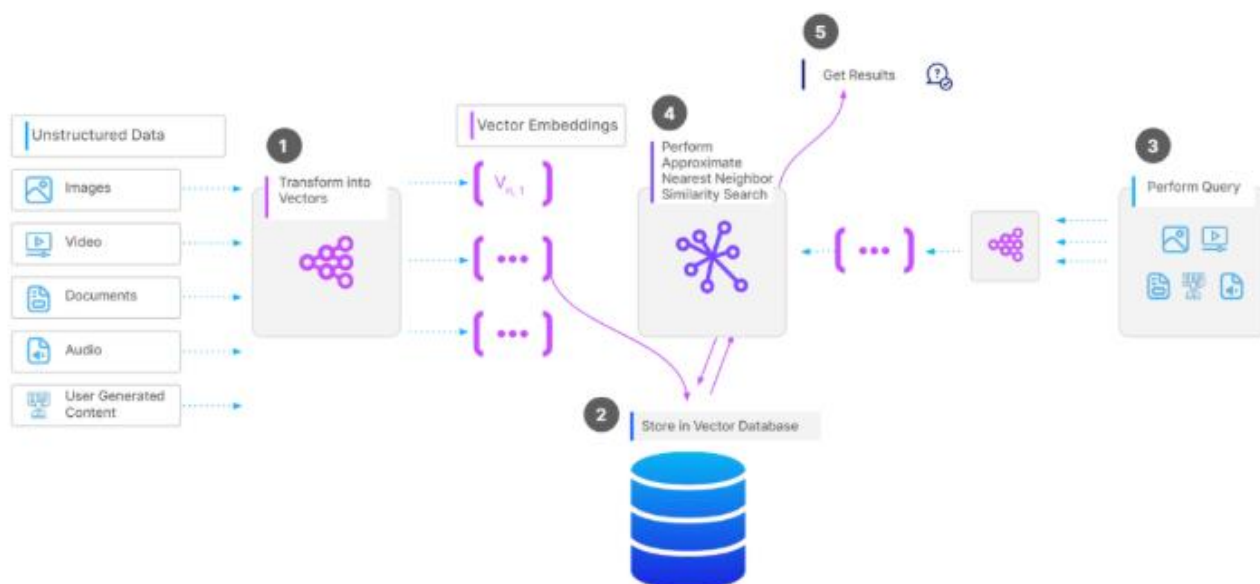


Figure 1 : Architecture générale d'un système vectoriel

(Vector Search Workflow s.d.)

Comme le montre la Figure 1, l'architecture d'un système vectoriel s'articule autour de plusieurs étapes clés. Les données non structurées constituent la source d'entrée du système et sont transformées en vecteurs numériques à l'aide de modèles d'*embedding*. Ces vecteurs sont ensuite stockés dans une base de données vectorielle, où ils sont indexés afin de permettre des recherches rapides.

Lorsqu'une requête est soumise, elle est vectorisée selon le même processus, puis comparée aux vecteurs stockés à l'aide d'algorithmes de recherche de plus proches voisins, tels que les méthodes *Approximate Nearest Neighbors*. Cette architecture permet d'assurer un compromis efficace entre performance et précision, ce qui la rend particulièrement adaptée aux cas d'usage en intelligence artificielle tels que la recherche sémantique ou les systèmes de recommandation.

Les systèmes vectoriels adoptent généralement une **architecture distribuée** et optimisée pour les requêtes de similarité. Il combine généralement :

- **Stockage spécialisé** : basé sur des moteurs optimisés pour gérer des vecteurs de haute dimension. Certains systèmes s'appuient sur des bases relationnelles étendues (ex. : PostgreSQL avec pgvector), tandis que d'autres reposent sur des moteurs natifs (Weaviate, Pinecone).
- **Indexation avancée** : intégration de structures ANN (HNSW, IVF, etc.) pour accélérer la recherche.
- **Parallélisation et distribution** : afin de gérer des volumes massifs de données, plusieurs bases vectorielles adoptent une architecture distribuée avec *sharding* et réplication.

- **Intégration IA-native** : certains moteurs incluent directement des connecteurs avec des modèles d'apprentissage automatique (ex. Oracle AI Vector Search intégré à Oracle Database 23ai).

2.3 Domaine d'utilisation des bases de données vectorielles

i. Traitement automatique du langage naturel (NLP)

Les vecteurs sont essentiels dans des tâches comme :

- **Recherche sémantique** : retrouver un document pertinent en fonction d'une requête en langage naturel.
- **Question Answering** : associer une question utilisateur à la réponse la plus proche dans une base documentaire.
- **Chatbots avancés** : utiliser des vecteurs pour enrichir la mémoire contextuelle et améliorer la pertinence des réponses. (RAG)

ii. Vision par ordinateur

Dans le domaine de l'analyse d'images et de vidéos :

- **Recherche d'images par similarité** : retrouver des images proches d'une requête donnée (par exemple, trouver des produits similaires dans un catalogue e-commerce).
- **Détection et classification d'objets** : utiliser des vecteurs issus de réseaux convolutifs pour indexer et retrouver des objets visuels.
- **Analyse vidéo** : segmenter et retrouver des séquences pertinentes dans des contenus multimédias.

iii. Systèmes de recommandation

- **Recommandations personnalisées** : comparer le profil vectoriel d'un utilisateur avec celui d'items (films, produits, musiques).
- **Détection d'anomalies** : identifier des comportements atypiques en mesurant leur éloignement dans l'espace vectoriel.
- **Publicité ciblée** : associer vecteurs de préférences utilisateur et vecteurs de campagnes publicitaires.

3 État de l'art sur les moteurs de bases de données vectorielles

L'évolution massif des données non structurées (textes, images, vidéos, signaux audio) conjugué aux avancées des modèles d'intelligence artificielle a transformé les méthodes de stockage et d'interrogation de l'information. Les bases de données vectorielles, ou moteurs de recherche vectorielle, se sont imposées comme une réponse incontournable à ce besoin. Leur rôle principal consiste à représenter les données sous forme de vecteurs numériques (embeddings), à les indexer efficacement et à permettre des recherches par similarité sémantique.

Contrairement aux bases relationnelles classiques, qui gèrent essentiellement des données structurées, les moteurs de bases de données vectorielles exploitent des algorithmes avancés d'indexation et de recherche approximative (ANN). Ces approches garantissent une exploration rapide et scalable de milliards de vecteurs tout en préservant une qualité de correspondance élevée.

Dans ce contexte, plusieurs solutions ont émergé, portées par de grands acteurs de l'industrie et de la recherche, chacune proposant une combinaison spécifique d'architectures, de performances et de fonctionnalités. Cette section propose un état de l'art comparatif de moteurs emblématiques, à savoir **Oracle AI Vector**, **pgvector**, **Weaviate** et **FAISS**, en s'appuyant sur des critères rigoureusement définis.

3.1 Critères d'évaluation

Afin d'évaluer et de comparer les moteurs de bases de données vectorielles, plusieurs critères pertinents ont été retenus :

1. **Critères conceptuels et architecturaux**

Ce critère examine les concepts sous-jacents, le modèle de stockage adopté (natif ou extension), ainsi que l'architecture logicielle (centralisée, distribuée, cloud-native).

2. **Indexation et recherche vectorielle**

Analyse des mécanismes d'indexation proposés (HNSW, IVF etc.), de la gestion de la recherche exacte vs approximative, ainsi que des stratégies d'optimisation.

3. **Performance**

Mesure de la scalabilité, du temps de réponse, de la capacité à gérer de grands volumes de vecteurs et de la consommation de ressources.

4. **Fonctionnalités avancées**

Évaluation des services complémentaires : support multimodal (texte, image, audio), gestion des métadonnées, intégration avec des modèles de machine learning, fonctionnalités analytiques et support du temps réel.

5. **Interopérabilité et intégration**

Capacité du moteur à s'intégrer dans des pipelines de données existants, compatibilité avec d'autres bases de données ou frameworks IA, support d'API et connecteurs.

6. **Domaine d'usage**

Illustration des principaux domaines d'application (recherche sémantique, recommandation, détection de fraude, vision par ordinateur, assistants IA, etc.).

7. **Algorithmes de recherche de similarité**

Description des méthodes employées pour mesurer la proximité entre vecteurs (cosinus, L2, dot product, etc.) et prise en charge de métriques multiples.

8. **Aspects pratiques**

Prise en compte de la facilité d'installation, documentation, communauté d'utilisateurs, support commercial éventuel et coût d'adoption (open source vs propriétaire).

3.2 **Comparaison des différentes Base de données Vectorielles**

3.2.1 Oracle AI Vector

Critères conceptuels et architecturaux

Oracle AI Vector est une extension native intégrée à Oracle Database 23ai. Elle repose sur l'architecture multimodèle d'Oracle, permettant de gérer simultanément données relationnelles, JSON, spatiales et désormais vectorielles. Contrairement à certains moteurs spécialisés, Oracle ne crée pas une base dédiée mais enrichit sa base relationnelle avec le support des vecteurs via le type VECTOR.

Indexation et recherche vectorielle

Oracle propose des index vectoriels basés sur HNSW (Hierarchical Navigable Small World) pour la recherche approximative de similarité (ANN). Les vecteurs sont stockés et indexés comme des colonnes natives, ce qui permet des requêtes SQL classiques enrichies d'opérateurs tels que COSINE_DISTANCE, DOT_PRODUCT ou EUCLIDEAN_DISTANCE.

Performance

Les performances sont optimisées grâce à l'intégration directe dans le moteur Oracle, ce qui évite des appels externes. Oracle exploite aussi ses fonctionnalités de parallélisation et de partitionnement pour accélérer les requêtes sur de très grands volumes de données.

Fonctionnalités avancées

- Recherche hybride (combiner filtres relationnels + recherche vectorielle).

- Intégration avec Oracle Machine Learning et des modèles externes.
- Sécurité avancée (Data Masking, Audit, RBAC) héritée d'Oracle.
- Support pour RAG (Retrieval Augmented Generation) avec LLMs.

Interopérabilité et intégration

Compatible avec tout l'écosystème Oracle (APEX, GoldenGate, Data Pump, Big Data SQL). Oracle AI Vector se connecte aussi à des pipelines IA via Python, Java, REST API et supporte OCI AI Services.

Domaine d'usage

- Recherche multimodale (texte, image, audio).
- Génération augmentée par la recherche (chabots).
- Intégration dans les entrepôts de données d'entreprise.

Algorithmes de recherche de similarité

Support natif de la **distance cosinus**, de la **distance euclidienne** et du **produit scalaire**.

Aspects pratiques

L'installation se fait automatiquement avec Oracle 23ai. Aucune infrastructure externe n'est nécessaire. Le coût reste lié au modèle de licence Oracle, ce qui peut être un frein pour des usages académiques ou petites entreprises.

(Oracle Enterprise 2024)

3.2.2 Pgvector

Critères conceptuels et architecturaux

Pgvector est une extension de PostgreSQL ajoutant un type de données vector. Elle repose sur la philosophie open source et l'architecture client-serveur PostgreSQL.

Indexation et recherche vectorielle

Supporte plusieurs index :

- IVFFlat (Inverted File Index) pour la recherche ANN.

- HNSW depuis les versions récentes.
- Scans séquentiels pour de petits jeux de données.

Performance

Moins performant que des moteurs spécialisés comme FAISS, mais suffisant pour des ensembles de taille moyenne (< 100M vecteurs). La performance dépend fortement du tuning de PostgreSQL (indexation, mémoire).

Fonctionnalités avancées

- Intégration SQL native.
- Support des distances : cosine, L2 (euclidean), inner product.
- Peut-être combiné avec toutes les extensions PostgreSQL (PostGIS, TimescaleDB, etc.).

Interopérabilité et intégration

Très flexible car s'intègre dans tout environnement basé sur PostgreSQL. Compatible avec LangChain, LlamaIndex, et la plupart des frameworks IA.

Domaine d'usage

- Recherche sémantique dans des applications web.
- Stockage vectoriel léger pour projets IA open source.
- Prototypage rapide avec LLMs.

Algorithmes de recherche de similarité

Support : cosinus, euclidienne, inner product.

Aspects pratiques

Installation simple (CREATE EXTENSION pgvector). Open source et gratuit, avec une large communauté. Moins adapté à l'échelle industrielle très massive (>1 milliard de vecteurs).

(pgvector 2024)

3.2.3 Weaviate

Critères conceptuels et architecturaux

Weaviate est une base de données vectorielle native et open source conçue spécifiquement pour les applications IA. Elle repose sur une architecture distribuée orientée micro services.

Indexation et recherche vectorielle

Utilise principalement HNSW pour l'ANN. Permet une recherche hybride combinant BM25 (recherche full-texte) et vecteurs.

Performance

Optimisé pour la scalabilité horizontale. Peut gérer des milliards de vecteurs avec une recherche rapide grâce à l'index HNSW distribué.

Fonctionnalités avancées

- API GraphQL pour la recherche sémantique.
- Support des multimodaux (texte, image, audio, vidéo).
- Intégration directe de modèles de ML pour la vectorisation (OpenAI, HuggingFace).
- Support du **RAG** et de pipelines IA.

Interopérabilité et intégration

Facilement intégrable avec **LangChain**, **Transformers**, **OpenAI API**.
Propose un **hébergement managé (Weaviate Cloud Service)** ou installation on-premise.

Domaine d'usage

- Chatbots IA en production.
- Recherche multimodale sur de grands corpus.
- Applications temps réel (par ex. moteurs de recommandation).

Algorithmes de recherche de similarité

Supporte **cosinus**, **euclidienne**, **produit scalaire**.

Aspects pratiques

Solution clé en main avec API simple. Requiert cependant une infrastructure Kubernetes ou Docker pour un déploiement distribué.

(Weaviate 2023)

3.2.4 FAISS (Facebook AI Similarity Search)

Critères conceptuels et architecturaux

FAISS est une bibliothèque C++/Python développée par Meta AI, optimisée pour la recherche vectorielle et le clustering. Contrairement aux autres, ce n'est pas une base de données complète mais une librairie spécialisée.

Indexation et recherche vectorielle

Propose de nombreux index ANN :

- IVF
- PQ
- HNSW
- Flat

Performance

Extrêmement performant et optimisé GPU. Référence académique et industrielle pour l'ANN sur très grands volumes (>1 milliard vecteurs).

Fonctionnalités avancées

- Compression vectorielle pour économiser mémoire et stockage.
- Recherche GPU pour accélération massive.
- Outliers de clustering (k-means).

Interopérabilité et intégration

S'intègre via Python avec Numpy, PyTorch, HuggingFace. Peut-être combiné avec une base relationnelle pour le stockage des métadonnées.

Domaine d'usage

- Systèmes de recommandation à grande échelle.
- Recherche multimédia (images, vidéos).
- Expérimentation IA à grande échelle.

Algorithmes de recherche de similarité

Support d'une large gamme : cosinus, L2, produit scalaire.

Aspects pratiques

Très performant mais nécessite une gestion externe des métadonnées (pas une base clé-valeur complète).

Installation simple via `pip install faiss-cpu` ou `faiss-gpu`.

(Facebook research 2023) (Meta Platforms, Inc. and affiliates n.d.)

3.3 Tableau comparatif des moteurs de BD vectoriels

Critères	Oracle AI Vector	Pgvector	Weaviate	FAISS
Critères conceptuels et architecturaux	Intégré nativement à Oracle 23ai ; architecture relationnelle + vecteurs dans des colonnes dédiées ; support multi-modèle.	Extension PostgreSQL (open source) ; ajoute un type <code>vector</code> et des fonctions associées.	Base de données vectorielle cloud-native ; API REST/GraphQL ; pensée pour la recherche sémantique.	Librairie C++/Python développée par Meta ; pas une BD complète, mais un moteur spécialisé dans la recherche vectorielle.
Indexation et recherche vectorielle	Index vectoriels (HNSW, IVF, brute force) intégrés dans Oracle ; support des distances Cosine, Euclidienne, Inner Product.	Index IVFFlat, HNSW ; distances Euclidienne, Cosine, Inner Product.	Index HNSW optimisé pour la recherche sémantique ; support de filtres hybrides (mots-clés + vecteurs).	Index puissants (Flat, IVFFlat, IVFPQ, HNSW) ; spécialisé dans l'approximate nearest neighbor (ANN).
Performance	Haute performance grâce à l'intégration Oracle et au parallélisme ; adapté	Performances correctes mais dépendantes des ressources PostgreSQL ; efficace pour des	Optimisé pour scalabilité horizontale et cloud ; très rapide sur données non structurées.	Très haute performance sur millions à milliards de vecteurs ; optimisé GPU et CPU.

	aux très grands volumes (Big Data).	bases de taille moyenne.		
Fonctionnalités avancées	Combinaison SQL + vector search ; intégration avec données relationnelles, JSON, multimodèle (texte, image, vidéo).	Simple et léger ; peu de fonctionnalités avancées hors PostgreSQL standard.	Recherche multimodale (texte, image, audio) ; intégration NLP ; génération de schémas automatiques.	Support quantisation, compression, recherche ANN massive ; mais pas de gestion de transactions ni de données relationnelles.
Interopérabilité et intégration	Intégré à l'écosystème Oracle (PL/SQL, Big Data SQL, OCI) ; interfaçage Python, Java, etc.	Fonctionne dans l'écosystème PostgreSQL ; interfaçage Python, R, etc.	API REST, GraphQL, clients pour Python, Go, Java ; intégration avec modèles NLP (Hugging Face, OpenAI).	Disponible en C++/Python ; utilisé dans d'autres BD vectorielles (Milvus, Weaviate).
Domaine d'usage	Recherche multimodale (documents, images, vidéos) ; finance, santé, e-commerce ; intégration dans systèmes critiques.	Recherche sémantique et recommandation ; petites/moyennes entreprises ; intégration rapide dans applications PostgreSQL.	Recherche sémantique dans le cloud ; moteurs de recherche contextuels ; assistants IA multimodaux.	Recherche à grande échelle (images, vidéos, embeddings) ; utilisé en recherche académique et industrielle (reco, NLP, vision).
Algorithmes de similarité	Cosine, Euclidienne (L2), Inner Product.	Cosine, Euclidienne, Inner Product.	Principalement Cosine et L2 ; via HNSW.	Large choix (L2, Cosine, IP) + variantes optimisées ANN.

Tableau 1 : Comparatif des moteurs vectoriels

Au terme de cette étude comparative des principaux moteurs de bases de données vectorielles, il apparaît que chaque solution possède des avantages spécifiques :

- Les moteurs spécialisés comme **Weaviate** offrent une grande flexibilité et des performances élevées dans des environnements cloud natifs.
- Des solutions open source comme **FAISS** se distinguent par leur rapidité et leur adaptabilité, mais exigent une expertise technique importante pour l'intégration et la maintenance.
- Les SGBD traditionnels enrichis de fonctionnalités vectorielles (PostgreSQL + pgvector) séduisent par leur simplicité d'usage mais montrent des limites de performance à très grande échelle.

En revanche, **Oracle Database 23ai** se démarque par une approche intégrée et robuste :

- Elle combine la puissance d'un **SGBD relationnel mature** avec des **capacités vectorielles natives**, permettant une gestion unifiée des données relationnelles, textuelles et vectorielles.
- Son **architecture distribuée et résiliente** garantit une haute disponibilité et une montée en charge efficace.
- Les fonctionnalités avancées, telles que la recherche hybride (vecteur + SQL), l'intégration aux modèles IA en fait un moteur adapté aux environnements critiques.
- Enfin, Oracle s'intègre parfaitement dans un **écosystème Big Data** (Hadoop, Spark, Data Lake), ce qui répond à la vision d'un stockage et d'une exploitation multi-sources.

Ainsi, au vu de ces critères techniques, fonctionnels et pratiques, notre choix se porte sur **Oracle Database 23ai** comme moteur principal pour la mise en œuvre et l'expérimentation de bases de données vectorielles. Ce choix est motivé par la volonté de disposer d'une solution **robuste, évolutive et alignée sur les standards industriels**, tout en garantissant la possibilité d'intégrer des cas d'usage concrets tels que la recherche sémantique, les systèmes de recommandation ou encore l'analyse multimodale.

4 Oracle Data base 23ai et la gestion des bases de données vectorielles

Ce chapitre présente le cadre technique et architectural d'Oracle Database 23ai dans le contexte de la gestion et de l'exploitation des bases de données vectorielles. Il vise à expliciter les objectifs poursuivis par Oracle, les composants et mécanismes mis à disposition, ainsi que les choix architecturaux permettant d'assurer performance, scalabilité et résilience. Ce chapitre constitue une base conceptuelle et technique indispensable avant les phases d'installation, de prise en main et d'expérimentation présentées dans les chapitres suivants.

4.1 Objectifs et positionnement d'Oracle Database 23ai

L'intégration des bases de données vectorielles dans Oracle Database 23ai répond à un objectif majeur : permettre la gestion et l'interrogation efficace de données complexes et multimodales, telles que les textes, les images, les vidéos ou encore les embeddings générés par des modèles d'intelligence artificielle. L'essor des modèles de langage de grande taille (LLMs) et des réseaux de neurones profonds a en effet rendu indispensable la capacité à stocker et à rechercher directement des représentations vectorielles au sein des systèmes de gestion de bases de données.

Oracle 23ai vise ainsi à rapprocher les capacités de l'intelligence artificielle des mécanismes traditionnels des bases de données relationnelles, en proposant une solution intégrée permettant d'éviter la multiplication de systèmes spécialisés externes. Les objectifs principaux poursuivis par cette approche sont les suivants :

- Permettre le **stockage natif des vecteurs**, sous forme dense ou clairsemée (sparse), au sein des tables Oracle ;
- Offrir une **recherche vectorielle performante**, reposant sur différentes métriques de similarité telles que la distance cosinus ou la distance euclidienne ;
- Assurer une **intégration transparente avec l'écosystème Oracle**, notamment via SQL, PL/SQL et les outils analytiques existants ;
- Fournir un **support adapté aux cas d'usage IA et ML**, incluant le traitement automatique du langage naturel, la vision par ordinateur et les systèmes de recommandation ;
- Garantir des **performances élevées et une bonne scalabilité**, grâce à des mécanismes d'indexation spécialisés et à l'architecture distribuée d'Oracle.

Ce positionnement fait d'Oracle Database 23ai une solution hybride, combinant les avantages des bases relationnelles traditionnelles et ceux des bases de données vectorielles spécialisées.

4.2 Composants et mécanismes vectoriels d'Oracle Database 23ai

Oracle Database 23ai met à disposition un ensemble de composants et de mécanismes permettant la gestion complète du cycle de vie des données vectorielles, depuis leur stockage jusqu'à leur exploitation analytique.

Le premier élément central est le **type de données VECTOR**, qui permet de stocker directement des vecteurs numériques de dimension variable au sein des tables Oracle. Ce type natif facilite l'intégration des embeddings issus de modèles d'IA sans nécessiter de transformations complexes ou de solutions de stockage externes.

Oracle propose également des **fonctions analytiques dédiées**, permettant le calcul de distances entre vecteurs, la normalisation des représentations et l'exécution de requêtes de similarité directement en SQL. Ces fonctions rendent possible l'implémentation de recherches de type k plus proches voisins (k-NN) de manière déclarative.

En complément, Oracle 23ai supporte des **mécanismes d'indexation**, incluant des approches exactes et approximatives, afin d'optimiser les performances des requêtes de similarité sur de grands volumes de vecteurs. Ces index spécialisés constituent un levier essentiel pour garantir des temps de réponse compatibles avec des applications en temps réel.

L'intégration avec les **outils d'intelligence artificielle et de machine learning** constitue un autre pilier important. Oracle permet l'interaction avec des environnements externes tels que Python, ainsi que l'exploitation de modèles NLP ou de vision par ordinateur pour la génération et l'analyse des vecteurs.

Enfin, Oracle Database 23ai conserve les fonctionnalités avancées de **sécurité et de gouvernance** propres aux bases de données Oracle, incluant l'authentification, l'audit, le chiffrement des données et les mécanismes de contrôle d'accès. Des **outils d'administration**, notamment sous forme de packages PL/SQL, facilitent la gestion et la supervision des données vectorielles au sein du système.

4.3 Architecture distribuée, scalabilité et résilience

L'architecture d'Oracle Database 23ai repose sur des principes éprouvés de distribution et de résilience, adaptés aux exigences des charges de travail modernes intégrant des données vectorielles. Cette architecture permet de répondre aux besoins de montée en charge, de disponibilité et de continuité de service.

Oracle 23ai supporte la **scalabilité verticale**, par l'augmentation des ressources matérielles disponibles sur un même nœud, ainsi que la **scalabilité horizontale**, via la répartition des charges sur plusieurs nœuds au sein d'une architecture distribuée. Cette flexibilité est essentielle pour gérer des volumes croissants de vecteurs et des requêtes de similarité intensives.

Les mécanismes de résilience intégrés à Oracle assurent une tolérance aux pannes et une haute disponibilité des services. Ils incluent notamment la réplication des données, la gestion des défaillances et des mécanismes de reprise automatique, garantissant la continuité des opérations même en cas d'incident.

Oracle Database 23ai s'intègre également dans des environnements **cloud et hybrides**, permettant de déployer des architectures adaptées aussi bien à des contextes on-premise qu'à des infrastructures cloud ou Edge. Cette capacité d'intégration facilite l'adoption des fonctionnalités vectorielles dans des systèmes distribués à grande échelle.

(Oracle Enterprise 2024)

5 Installation d'Oracle 23AI

5.1 Contexte et enjeux

Dans un contexte où l'intelligence artificielle (IA), le traitement du langage naturel (NLP) et l'analyse vectorielle deviennent des piliers de la transformation numérique, les bases de données traditionnelles évoluent pour intégrer ces nouvelles exigences. Oracle Database, reconnu comme l'un des SGBD les plus robustes et évolutifs, a su suivre cette tendance en introduisant dans sa version 23ai une prise en charge native des vecteurs.

Cette évolution ouvre la voie à des cas d'usage avancés dans le domaine de la **similarité sémantique**, de la **recherche vectorielle** ou encore de l'**intégration IA dans les bases de données d'entreprise**. La maîtrise d'un tel environnement constitue une étape stratégique pour tout projet explorant les bases de données vectorielles avec Oracle.

5.2 Objectifs de cette première phase

L'objectif principal de cette phase est de manipuler concrètement une base de données vectorielle avec Oracle Database 23ai, en explorant les fonctionnalités offertes par cette nouvelle version dans le cadre de traitements orientés IA et NLP.

Pour atteindre cet objectif central, plusieurs étapes techniques et méthodologiques doivent être réalisées :

- **Installer et configurer Oracle 23ai** sur un poste local Windows, en s'assurant que l'environnement est pleinement opérationnel pour les futures manipulations vectorielles ;
- **Comprendre l'organisation technique d'Oracle** post-installation (services, ports, architecture multi-tenant, etc.), pour maîtriser le fonctionnement du système ;
- **Créer une structure adaptée au stockage de vecteurs**, incluant un schéma personnalisé (ex. `Ryan`) et des tables capables de contenir des représentations vectorielles (ex. `nlp_vectors`) ;
- **Établir une passerelle entre Python et Oracle**, via des bibliothèques telles qu'`oracledb`, afin de permettre l'insertion et l'interrogation de vecteurs générés avec des modèles préentraînés (`sentence-transformers`, `all-MiniLM-L6-v2`, etc.) ;
- **Effectuer des premières opérations de test** sur les vecteurs insérés (recherches par similarité, distances euclidiennes, etc.) pour valider la faisabilité et le bon fonctionnement du pipeline complet.

Ces travaux constituent la **base expérimentale** du mémoire, et préparent les étapes suivantes : évaluation des performances, intégration potentielle dans des systèmes plus larges (data lake, pipelines IA), et comparaison avec d'autres solutions vectorielles.

5.3 Prérequis matériels et logiciels

5.3.1 Environnement matériel utilisé

Dans le cadre de ce projet, nous avons réalisé l'installation d'**Oracle Database 23ai** sur un ordinateur personnel, équipé des caractéristiques suivantes :

- **Système d'exploitation** : Windows 10, 64 bits
- **Processeur** : Intel Core i7
- **Mémoire vive** : 16 Go
- **Disque de stockage** : SSD de 512 Go
- **Résolution d'écran** : 1366 x 768 pixels

Cette configuration, bien que modeste en résolution d'affichage, s'est révélée largement suffisante pour exécuter Oracle de manière stable. Le processeur i7 associé à 16 Go de RAM m'a permis de manipuler la base sans ralentissements notables.

5.3.2 Choix du system d'exploitations

Le système d'exploitation choisi pour cette prise en main d'Oracle Database est **Windows 10**. Ce choix s'explique par plusieurs raisons pratiques : il s'agit d'un environnement largement utilisé, stable, compatible avec la majorité des outils Oracle, et déjà installé sur la machine utilisée pour cette expérimentation. Windows 10 offre un bon compromis entre accessibilité, performance et compatibilité logicielle, ce qui en fait une plateforme idéale pour les phases d'installation, de configuration et de test.

Bien que Windows 10 soit doté d'une interface graphique conviviale, j'ai volontairement opté pour une interaction en ligne de commande avec les outils Oracle. Cela m'a permis de mieux explorer le fonctionnement interne du SGBD sans recourir à des outils graphiques comme SQL Developer ou DBeaver. Ainsi, l'ensemble des opérations de l'installation à la gestion de la base a été réalisé à travers des utilitaires tels que `sqlplus`, `lsnrctl` ou `dbca`.

Ce mode de fonctionnement, bien qu'exécuté sur un système Windows, nous a offert une expérience proche de celle rencontrée dans des environnements serveurs professionnels, souvent basés sur des systèmes Linux. Il a également facilité une compréhension plus fine des dépendances entre les composants Oracle et le système d'exploitation hôte.

5.3.3 Téléchargement

Nous avons téléchargé Oracle Database 23ai depuis le site officiel d'Oracle, en sélectionnant la version Windows 64 bits. Il s'agissait d'un fichier ZIP contenant tous les éléments nécessaires à l'installation.

Contrairement aux recommandations souvent données de créer un répertoire d'installation personnalisé, j'ai décidé de laisser **le chemin d'installation par défaut proposé par Oracle**, ce qui a installé Oracle dans le dossier :

C:\app\Dell\product\23ai\dbhomeFree

Ce chemin, bien que long, n'a pas posé de problème lors de l'exécution ou de la configuration de la base.

5.3.4 Validation des prérequis

Avant de procéder à l'installation, je me suis assuré que les conditions minimales requises étaient réunies :

- Mon système Windows était en 64 bits
- J'avais les **droits administrateurs** nécessaires pour lancer l'installation
- Les ports réseau standards (notamment le **1521** utilisé par le listener) n'étaient pas occupés
- Aucun produit Oracle n'était installé préalablement sur la machine
- Mon antivirus n'a pas bloqué le processus d'installation

La validation de ces points a permis une installation sans erreur majeure.

5.3.5 Bilan de la phase préparatoire

Cette étape a permis de définir l'environnement de travail basé sur Windows 10. Aucun outil graphique n'a encore été installé, mais l'utilisation de la ligne de commande a été privilégiée pour une meilleure compréhension du fonctionnement interne d'Oracle. L'installation de **SQL Developer** reste envisageable pour simplifier certaines opérations dans les étapes suivantes.

5.4 Choix de la version : Oracle Database 23ai

Oracle 23ai est une version orientée vers l'intelligence artificielle et l'automatisation. Elle apporte des innovations majeures telles que :

- Un **moteur vectoriel** intégré permettant de stocker, interroger et comparer des vecteurs,
- Une **meilleure compatibilité avec les environnements hybrides et cloud**,
- Des performances améliorées pour les opérations SQL, PL/SQL et les appels IA,
- Une architecture renforcée pour les bases de données autonomes.

Cette version est donc particulièrement adaptée à l'exploration des fonctionnalités vectorielles que nécessite ce mémoire.



Fig 1 : Logo Oracle Database 23ai

(Get Ready for Oracle Database 23ai s.d.)

5.4.1 Téléchargement de l'archive Oracle

Je me suis rendu sur la page officielle d'Oracle (<https://www.oracle.com/database/free/get-started>) pour récupérer le fichier d'installation. Le téléchargement consistait en une **archive compressée au format ZIP** contenant un installeur auto-extractible. La taille du fichier avoisinait les **1,5 Go**, ce qui nécessitait une connexion internet stable.

5.4.2 Préparation avant exécution

Avant de lancer l'installateur, j'ai effectué les vérifications suivantes :

- J'ai exécuté le programme d'installation en tant qu'administrateur
- J'ai désactivé temporairement mon antivirus pour éviter qu'il n'interfère avec les opérations de copie de fichiers système,
- J'ai vérifié que les **ports 1521** utilisé par Oracle Net Listener étaient disponibles sur ma machine en exécutant la commande :

```
netstat -an | findstr 1521
```

Aucune occupation préalable de ces ports n'a été détectée, ce qui m'a permis de poursuivre sereinement

```
C:\Users\DELL>netstat -an | findstr 1521
TCP      0.0.0.0:1521          0.0.0.0:0             LISTENING
TCP      172.17.96.1:1521      172.17.96.1:52700     ESTABLISHED
TCP      172.17.96.1:52700     172.17.96.1:1521     ESTABLISHED
TCP      [::]:1521            [::]:0                LISTENING
```

5.5 Installation et post-installation

5.5.1 Démarrage de l'installation

Une fois le programme d'installation lancé, j'ai suivi le processus en sélectionnant l'option **“Desktop Class”**, qui correspond à une installation monutilisateur, simplifiée, idéale pour un usage local et non distribué. Cette option configure automatiquement :

- Un **container database (CDB)** avec un **pluggable database (PDB)** par défaut,
- Un listener Oracle (`LISTENER`) sur le port 1521,
- Les utilisateurs administratifs `SYS` et `SYSTEM`.

L'assistant m'a demandé de définir un mot de passe pour ces comptes. J'ai choisi un mot de passe respectant la politique de complexité d'Oracle (majuscule, minuscule, chiffre, caractère spécial) pour éviter toute erreur de validation.

5.5.2 Structure des dossiers et chemins d'installation

Comme précisé dans la section précédente, j'ai accepté le chemin d'installation par défaut, qui a automatiquement créé la hiérarchie suivante :

```
C:\app\Dell\product\23ai\dbhomeFREE\
```

Dans ce répertoire, Oracle installe tous les composants essentiels :

- Les binaires (`sqlplus.exe`, `lsnrctl.exe`, etc.),
- Les fichiers de configuration (`sqlnet.ora`, `listener.ora`, `tnsnames.ora`),
- Le répertoire `network/admin` où se trouvent les paramètres réseau,
- Le répertoire `database`, contenant les fichiers initiaux (`init.ora`, `spfile`, etc.).

Cette organisation m'a permis de repérer rapidement les éléments nécessaires pour la configuration et la gestion de la base.

5.5.3 Services Oracle installés

Après l'installation, plusieurs **services Windows** ont été automatiquement créés. J'ai pu les visualiser via la commande :

```
services.msc
```

Les principaux services installés étaient :

- **OracleServiceORCLCDB** : service principal de la base de données,
- **OracleOraDB23Home1TNSListener** : listener de connexion (port 1521),
- **OracleVssWriterORCLCDB** : service de support pour les sauvegardes Windows.

Tous ces services étaient lancés automatiquement, ce qui m'a permis de commencer à travailler sans configuration réseau supplémentaire.

5.5.4 Premier test de connexion avec SQL*Plus

Pour m'assurer que l'installation avait bien fonctionné, j'ai effectué un premier test de connexion en utilisant l'utilitaire `sqlplus`, qui est un outil en ligne de commande fourni nativement avec Oracle.

Depuis une invite de commande Windows, j'ai tapé :

```
sqlplus / as sysdba
```

Cette commande m'a permis d'ouvrir une session administrative locale sans mot de passe, car elle utilise l'authentification OS si l'utilisateur Windows est administrateur.

Le prompt `SQL>` s'est affiché, confirmant que ma base était bien opérationnelle.

J'ai ensuite testé quelques commandes simples, par exemple :

```
SELECT name FROM v$database;
```

Cela m'a permis de confirmer que la base installée était bien `FREE`, le nom par défaut du container database créé par l'installateur.

```

C:\Users\DELL>sqlplus / as sysdba

SQL*Plus: Release 23.0.0.0.0 - Production on Dim. Juil. 20 14:15:56 2025
Version 23.8.0.25.04

Copyright (c) 1982, 2025, Oracle. All rights reserved.

Connecté à :
Oracle Database 23ai Free Release 23.0.0.0.0 - Develop, Learn, and Run for Free
Version 23.8.0.25.04

SQL> SELECT name FROM v$database;

NAME
-----
FREE

SQL>

```

5.5.5 Connexion à la base PDB

Bien que la base CDB ait été automatiquement créée, Oracle 23ai installe également un **Pluggable Database (PDB)** nommé **FREEPDB1**.

Pour me connecter à cette PDB en mode SQL*Plus, j'ai utilisé la commande suivante :

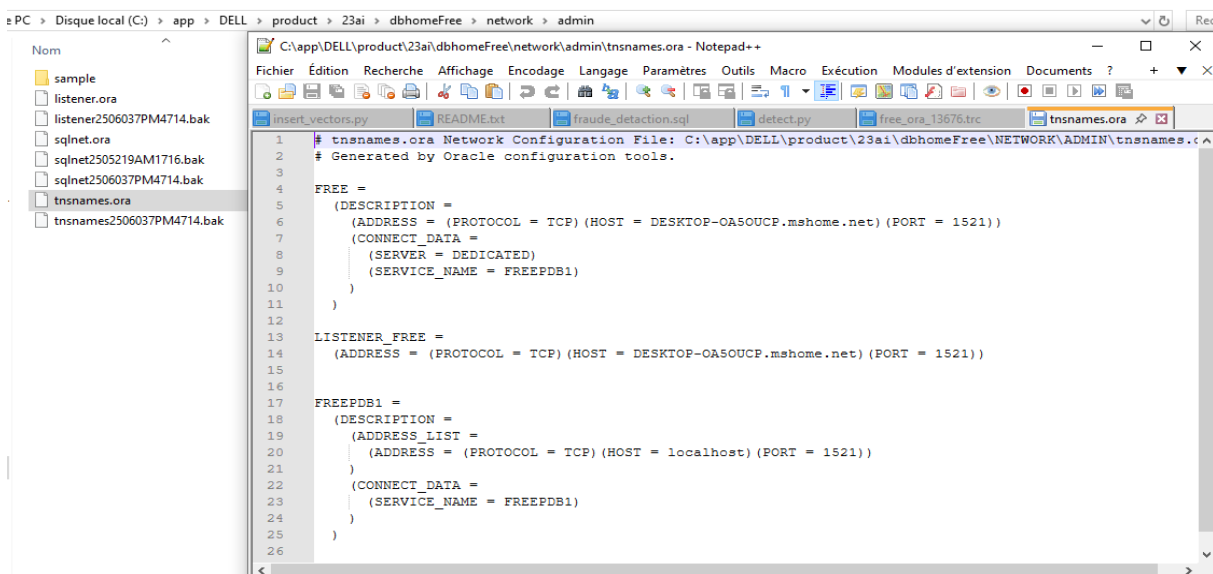
```
sqlplus sys@FREEPDB1 as sysdba
```

Et pour les connexions classiques :

```
sqlplus system@FREEPDB1
```

En cas d'erreur TNS, j'ai vérifié que le fichier `tnsnames.ora` contenait bien une entrée pour `FREEPDB1`, généralement située dans :

`C:\app\DELL\product\23ai\dbhomeFree\network\admin`



5.5.6 Résultat de l'installation

L'installation et la configuration de base ont été réalisées avec succès, sans aucune erreur majeure. À ce stade, j'étais capable de :

- Démarrer et arrêter la base (`startup`, `shutdown` via SQL*Plus),
- Consulter les vues dynamiques (`v$instance`, `v$database`, `dba_users...`),
- Me connecter aux deux environnements (CDB et PDB),
- Vérifier les paramètres initiaux de configuration (`show parameter`).

Cette base opérationnelle constituait le socle pour les expérimentations que j'ai menées par la suite (création d'objets, utilisateurs, requêtes, etc.).

6 MANIPULATION ET PRISE EN MAIN D'ORACLE 23ai

6.1 Manipulations de base : création d'utilisateurs, tables, insertion de données et requêtes SQL

Après avoir installé Oracle Database 23ai avec succès et confirmé son bon fonctionnement via la ligne de commande, j'ai entrepris une série de manipulations de base pour m'approprier progressivement l'environnement Oracle. Ces premières étapes m'ont permis de comprendre la structure relationnelle d'Oracle, le fonctionnement des droits d'accès, et les commandes SQL les plus courantes.

6.1.1 Création d'un utilisateur Oracle personnalisé

Afin de ne pas utiliser les comptes système par défaut (`sys`, `system`), j'ai commencé par me placer dans le conteneur **FREEPDB1** avec la commande :

```
ALTER SESSION SET CONTAINER=FREEPDB1;
```

Et en suit créer un utilisateur spécifique dédié à mes manipulations.

La commande exécutée dans SQL*Plus était la suivante :

```
CREATE USER ryan IDENTIFIED BY ryan23;
```

Ensuite, j'ai attribué à cet utilisateur les privilèges nécessaires pour créer des objets (tables, vues, index...) et exécuter des requêtes :

```
GRANT CONNECT, RESOURCE TO ryan;
```

Ce nouvel utilisateur m'a servi tout au long de mes tests, dans un environnement contrôlé et isolé.

```
SQL> ALTER SESSION SET CONTAINER = FREEPDB1;
Session modifiée.
SQL> CREATE USER Ryan IDENTIFIED BY ryan23 ;
Utilisateur créé.
```

6.1.2 Connexion avec l'utilisateur créé

Toujours via la ligne de commande, j'ai établi une nouvelle session avec cet utilisateur :

```
sqlplus ryan/ryan23@localhost :1521/FREEPDB1
```

Ou si on se trouve déjà dans une session SQL

```
connect ryan/ryan23@FREEPDB1
```

Une fois connecté, j'ai pu commencer mes expérimentations dans mon propre schéma.

6.1.3 Création d'une table simple

Pour tester la création de tables et l'insertion de données, j'ai d'abord défini une structure simple destinée à contenir des phrases et des identifiants :

```
CREATE TABLE nlp_sentences (  
    id NUMBER PRIMARY KEY,  
    phrase VARCHAR2(500)  
);
```

Cette table contient deux colonnes : un identifiant numérique unique et une phrase textuelle. Oracle impose la définition explicite des types, ce qui m'a obligé à réfléchir à la taille maximale des champs et au type exact à utiliser.

6.1.4 Insertion de données

Une fois la table créée, j'ai inséré mes premières données manuellement via la commande `INSERT` :

```
INSERT INTO nlp_sentences (id, phrase) VALUES (1, 'Oracle est une base de données  
robuste.') ;  
INSERT INTO nlp_sentences (id, phrase) VALUES (2, 'La manipulation se fait en ligne de  
commande.') ;
```

```
INSERT INTO nlp_sentences (id, phrase) VALUES (3, 'SQL est un langage de requêtes puissant.')
```

Après insertion, je validé les changements avec :

```
COMMIT ;
```

Cela m’a permis de mieux comprendre la gestion des transactions dans Oracle, où une insertion n’est réellement enregistrée que si elle est validée par un commit explicite.

```
SQL> CREATE TABLE nlp_sentences (  
2     id NUMBER PRIMARY KEY,  
3     phrase VARCHAR2(500)  
4 );  
Table cre.  
SQL> INSERT INTO nlp_sentences (id, phrase) VALUES (1, 'Oracle est une base de donnees robuste.');
```

ID	PHRASE
1	Oracle est une base de donnees robuste.
2	La manipulation se fai en ligne de commande.
3	SQL est un langage de requetes puissant

```
1 ligne cre.  
SQL> INSERT INTO nlp_sentences (id, phrase) VALUES (2, 'La manipulation se fai en ligne de commande.');
```

ID	PHRASE
1	Oracle est une base de donnees robuste.
2	La manipulation se fai en ligne de commande.
3	SQL est un langage de requetes puissant

```
1 ligne cre.  
SQL> INSERT INTO nlp_sentences (id, phrase) VALUES (3, 'SQL est un langage de requetes puissant');
```

ID	PHRASE
1	Oracle est une base de donnees robuste.
2	La manipulation se fai en ligne de commande.
3	SQL est un langage de requetes puissant

```
1 ligne cre.  
SQL> commit;  
Validation effectue.  
SQL>
```

6.1.5 Requêtes SQL de lecture

Pour afficher les données insérées, j’ai utilisé la requête suivante :

```
SELECT * FROM nlp_sentences ;
```

Le résultat retourné était :

```
SQL> SELECT * FROM nlp_sentences;  
-----  
ID  
-----  
PHRASE  
-----  
1  
Oracle est tres puissant.  
2  
La manipulation se fai en ligne de commande.  
3  
SQL est un langage de requetes puissant
```

6.1.6 Mise à jour et suppression de données

J’ai également testé la modification et la suppression de lignes :

- **Mise à jour** d'un enregistrement :

```
UPDATE nlp_sentences SET phrase = 'Oracle est très puissant.' WHERE id = 1 ;  
COMMIT ;
```

- **Suppression** d'un enregistrement :

```
DELETE FROM nlp_sentences WHERE id = 2;  
COMMIT ;
```

Grâce à ces commandes, j'ai validé la maîtrise des opérations fondamentales sur les données.

6.1.7 Synthèse

Ces manipulations m'ont permis de :

- Créer un environnement utilisateur personnalisé dans Oracle,
- Comprendre les rôles, privilèges et la gestion des connexions via SQL*Plus,
- Créer et manipuler des tables classiques ainsi qu'une table vectorielle (`fraud_text_vectors`) dédiée au NLP,
- Insérer des données classiques en SQL
- Écrire des requêtes SQL conditionnelles

Cette phase m'a permis de développer une première maîtrise concrète de l'environnement Oracle, via les concepts théoriques de SQL.

6.2 Prise en main des bases de données vectorielles avec Oracle 23ai

6.2.1 Contexte et motivation

Après avoir pris en main les fonctionnalités de base de l'environnement Oracle, j'ai souhaité explorer certaines nouvelles capacités offertes par Oracle Database 23ai, notamment la création et manipulation des vecteurs. Cette phase avait pour objectif d'évaluer l'intérêt d'Oracle 23ai dans des cas d'usage modernes, en particulier dans le cadre du traitement de données vectorielles et de la data science.

6.2.2 Recherche vectorielle intégrée

Oracle 23ai introduit officiellement une prise en charge **native des vecteurs** dans son moteur relationnel. Cette capacité est essentielle pour le traitement de données sémantiques (NLP, embeddings, IA générative, etc.). J'ai pu expérimenter cette fonctionnalité dans mon cas d'usage basé sur le stockage de phrases vectorisées.

Points observés :

- Oracle permet la **définition explicite de types** `VECTOR(n)`, où n est le nombre de dimensions.
- L'opérateur `COSINE_DISTANCE` permet de comparer deux vecteurs stockés dans la base, facilitant la **recherche de similarité** directement en SQL.
- Le moteur prend également en charge la **recherche par plus proche voisin exact**.

6.2.3 Création d'une table plus avancée : vecteurs NLP

Dans le cadre d'un autre test lié au traitement du langage naturel (NLP), j'ai voulu stocker des vecteurs de caractéristiques. J'ai alors créé une table spécifique avec une colonne de type `VECTOR` (propre à Oracle 23ai) :

```
CREATE TABLE fraud_text_vectors (
    id NUMBER PRIMARY KEY,
    phrase VARCHAR2(1000),
    vector VECTOR(384)
);
```

Cette structure m'a permis de lier des phrases à leurs représentations vectorielles. Le type `VECTOR (384)` est une nouveauté d'Oracle 23ai et correspond à un tableau de 384 valeurs flottantes, généralement issu d'un modèle NLP comme `all-MiniLM-L6-v2`.

6.2.4 Intégration avec des modèles IA et NLP

Oracle 23ai propose une intégration facilitée avec des modèles d'IA externes, notamment dans un contexte cloud ou hybride. Bien que nous n'ayons pas utilisé directement Oracle AI Vector Search via OCI dans ce projet, Nous avons reproduire une démarche équivalente en local en **utilisant Python pour générer les vecteurs** à partir de phrases/images et les stocker dans Oracle.

Cela illustre la capacité d'Oracle à servir de **backend pour des workflows IA**, notamment :

- Génération de vecteurs avec `sentence-transformers`,
- Insertion dans Oracle via `Oracledb`,
- Requêtes SQL enrichies pour la recherche sémantique.

6.2.5 Insertion des données vectorielles en Python

Voici un exemple en Python avec `sentence-transformers` pour générer les vecteurs, et `Oracledb` pour l'insertion :

Une séquence `nlp_seq` a été créé et initialisé à 1 avec la commande suivante pour servir d’ID au record enregistré

`CREATE SEQUENCE nlp_seq START WITH 1 INCREMENT BY 1;`

```
import oracledb
import json
import numpy as np
from sentence_transformers import SentenceTransformer

# Connexion à la base de données
user = "ryan"
password = "ryan23"
dsn = "localhost:1521/FREEPDB1"

# Crée une instance du modèle de phrase
model = SentenceTransformer('all-MiniLM-L6-v2')

# Liste des phrases à insérer
phrases = [
    "Votre transaction de 100€ a été validée.",
    "Appel urgent ! Mettez à jour vos informations bancaires pour éviter le blocage de votre compte.",
    "Votre virement de 250€ est en cours.",
    "Alerte de sécurité : une activité inhabituelle a été détectée sur votre compte.",
    "Merci de nous avoir contactés, votre demande est en cours de traitement."
]

# Connexion à la base Oracle
connection = oracledb.connect(user=user, password=password, dsn=dsn)
cursor = connection.cursor()

# Boucle d'insertion
for phrase in phrases:
    # Génération du vecteur
    vector = model.encode(phrase)
    vector_str = json.dumps(vector.tolist()) # Conversion en texte JSON

    # Insertion dans la base
    sql = "INSERT INTO fraud_text_vectors (id, phrase, vector) VALUES (nlp_seq.NEXTVAL, :1, :2)"
    cursor.execute(sql, [phrase, vector_str])

# Validation de la transaction
connection.commit()

print("✅ Insertion terminée avec succès.")

# Fermeture
cursor.close()
connection.close()
```

```
C:\Users\DELL\Desktop\oracle_vector>python insert_vectors.py
2025-07-20 19:38:16.173076: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'cudart64_110.dll'; dlerror: cudart64_110.dll not found
2025-07-20 19:38:16.173482: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on your machine.
✅ Insertion terminée avec succès.

C:\Users\DELL\Desktop\oracle_vector>
```

6.2.6 Requêtes SQL sur la table (en Python)

Exemple pour récupérer les phrases les plus similaires à une requête.

```

import oracledb
import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Connexion Oracle
conn = oracledb.connect(user="ryan", password="ryan23", dsn="localhost:1521/FREEDB1")
cursor = conn.cursor()

# Requête pour récupérer les phrases et vecteurs
cursor.execute("SELECT id, phrase, vector FROM fraud_text_vectors")
rows = cursor.fetchall()

# Phrase à analyser
phrase_input = input("Entrez une phrase à analyser :\n→ ")
model = SentenceTransformer('all-MiniLM-L6-v2')
query_vector = model.encode(phrase_input)

# Calcul des similarités
results = []
for row in rows:
    id, phrase, vector_oracle = row
    vector = np.array(vector_oracle, dtype=np.float32)
    score = cosine_similarity([query_vector], [vector])[0][0]

    # Ajout de l'étiquette
    label = "FRAUDE" if score > 0.6 else "NORMALE"
    results.append((score, phrase, label))

# Affichage des 5 plus similaires
results.sort(reverse=True)
print("🔍 Résultats :")
for score, phrase, label in results[:5]:
    print(f"– {phrase} (score: {score:.4f}) → {label}")

cursor.close()
conn.close()

```

```

C:\Users\DELL\Desktop\oracle_vector>python detect.py
2025-07-20 19:40:40.668000: W tensorflow/stream_executor/platform/default/dso_loader.cc:64] Could not load dynamic library 'cudart64_110.dll'; dlopen: cudart64_110.dll not found
2025-07-20 19:40:40.668273: I tensorflow/stream_executor/cuda/cudart_stub.cc:29] Ignore above cudart dlerror if you do not have a GPU set up on your machine.
Entrez une phrase à analyser :
→ Cher client, nous avons détecté un accès non autorisé à votre compte.
🔍 Résultats :
- Nous avons détecté une anomalie sur votre prélèvement automatique. (score: 0.6532) → FRAUDE
- Nous avons bien enregistré votre changement d'adresse. (score: 0.5591) → NORMALE
- Nous avons besoin d'une vérification d'identité pour éviter la suppression de votre compte. (score: 0.5509) → NORMALE
- Nous avons bien reçu votre demande de remboursement. (score: 0.5407) → NORMALE
- Votre dossier a été transmis à notre service comptabilité. (score: 0.5258) → NORMALE
- Alerte : tentative de retrait non autorisé. Connectez-vous pour annuler. (score: 0.5186) → NORMALE
- Votre dossier a été transféré au service comptabilité. (score: 0.5019) → NORMALE

```

6.2.7 Résumé

Étape	Outil	Description
Création de l'utilisateur	SQL*Plus	Utilisateur <code>Ryan</code> avec droits
Création de la table	SQL	Table <code>fraud_text_vectors</code> avec colonne <code>VECTOR</code> (384)

Insertion	Python (Oracledb)	Insertion automatique de vecteurs encodés avec le model sentence transformer
Requêtage	Python	Recherche vectorielle par similarité (Cosines)

Tableau 2 : Récapitulatif de la prise en main d'oracle

6.2.8 Performances et observations

Sur la machine Windows 10 (i7, 16 Go RAM, SSD), Oracle 23ai s'est montré fluide pour des traitements de vecteurs en dimensions modérées (384 dimensions avec le modèle `all-MiniLM-L6-v2`). Le temps de réponse pour des requêtes de type `COSINE_DISTANCE` est resté inférieur à une seconde pour un petit volume de.

6.2.9 Synthèse

Cette phase m'a permis d'explorer les **fonctionnalités vectorielles natives d'Oracle Database 23ai**, en mettant en œuvre un cas d'usage complet allant de la génération de vecteurs sémantiques à leur insertion et interrogation via SQL.

Les principaux acquis sont :

- **Maîtrise de la structure** `VECTOR(n)` proposée par Oracle 23ai, adaptée au stockage d'embeddings issus de modèles NLP modernes ;
- **Création d'une base vectorielle simple**, structurée autour d'une table `fraud_text_vectors`, pour stocker des phrases et leurs vecteurs associés (384 dimensions) ;
- **Interopérabilité Oracle – Python**, illustrée par l'usage de `sentence-transformers` pour encoder les phrases et la bibliothèque `oracledb` pour l'insertion et la requête ;
- **Exécution de requêtes SQL vectorielles**, notamment des comparaisons basées sur la similarité cosinus (`COSINE_DISTANCE`) ;
- **Évaluation de performance locale**, démontrant la faisabilité de traitements vectoriels sur une machine personnelle sans infrastructure cloud.

Au-delà de l'aspect technique, cette étape confirme la **pertinence d'Oracle 23ai** pour des projets impliquant des données vectorielles, comme la recherche sémantique, les moteurs de recommandation, ou les systèmes d'assistance intelligente. Ces travaux constituent une **base expérimentale solide** pour la suite de mon mémoire, orientée vers la comparaison d'Oracle avec d'autres solutions vectorielles et leur intégration potentielle dans des architectures big data.

6.3 Conclusion de la phase d'installation et de prise en main

Cette première phase du mémoire a permis de mettre en place avec succès l'environnement technique nécessaire à l'exploitation des bases de données vectorielles avec Oracle Database 23ai sous Windows. De l'installation initiale à la prise en main des fonctionnalités de base, elle a posé les fondations indispensables aux futures expérimentations, notamment à travers la création de tables vectorielles et l'intégration de modèles NLP pour la génération d'embeddings textuels.

Les résultats obtenus sont encourageants : l'interopérabilité entre Oracle et les outils de traitement du langage naturel a été validée, démontrant la capacité de la base à stocker, interroger et exploiter des vecteurs directement via SQL. Cette étape constitue ainsi un prérequis essentiel pour le développement de cas d'usage avancés en recherche sémantique et en systèmes de recommandation.

Néanmoins, certaines limites ont été identifiées, notamment l'impossibilité de tester les fonctionnalités avancées de recherche vectorielle et les scénarios à grande échelle, en raison de contraintes techniques et de licence. Ces limitations ouvrent naturellement la voie aux travaux futurs, qui porteront sur l'exploration des mécanismes avancés d'Oracle 23ai, l'évaluation comparative avec d'autres moteurs vectoriels et l'intégration dans des architectures Big Data et Data Lake.

En définitive, cette phase a permis d'établir un socle technique solide tout en définissant clairement les perspectives nécessaires pour exploiter pleinement le potentiel d'Oracle 23ai dans des environnements orientés IA et traitement sémantique.

7 Intégration d'oracle 23ai et les bases de données vectorielles dans un environnement Bigdata/data lake

7.1 Introduction

L'explosion des données hétérogènes (textes, images, signaux, vidéos, etc.) dans le domaine de la santé exige des solutions capables de gérer à la fois des données relationnelles traditionnelles et des données non structurées. Dans ce contexte, **Oracle Database 23AI** s'impose comme une plateforme hybride permettant l'intégration de données vectorielles directement dans le moteur relationnel. Cette capacité permet non seulement d'exploiter les données transactionnelles, mais aussi de réaliser des analyses avancées basées sur l'intelligence artificielle (IA) et le Machine Learning.

L'objectif de ce chapitre est de présenter l'intégration d'Oracle 23AI avec les bases de données vectorielles dans un **environnement Big Data/Data Lake**, en s'appuyant sur l'architecture construite dans ce projet.

Avant d'aborder l'intégration d'Oracle 23ai dans un environnement Big Data, il est nécessaire de présenter l'application de référence qui servira de support expérimental à l'ensemble des analyses réalisées dans ce mémoire.

7.2 Application de référence et contexte expérimental

Afin d'illustrer concrètement l'intégration des bases de données vectorielles d'Oracle 23AI dans un environnement Big Data, une application de référence a été mise en place dans un contexte médical. Elle sert de support expérimental à l'ensemble des analyses réalisées dans ce mémoire et vise à simuler un système d'aide à la décision clinique basé sur la recherche de similarité.

Cette application s'appuie sur plusieurs jeux de données hétérogènes : des dossiers médicaux tabulaires issus d'un dataset Healthcare, des images rétinienne provenant du dataset Diabetic Retinopathy Diagnosis, ainsi que des fichiers CSV simulant des cas patients pour la recommandation de traitements. Ces sources illustrent la diversité des données rencontrées dans un Data Lake médical, combinant données structurées, textuelles et visuelles.

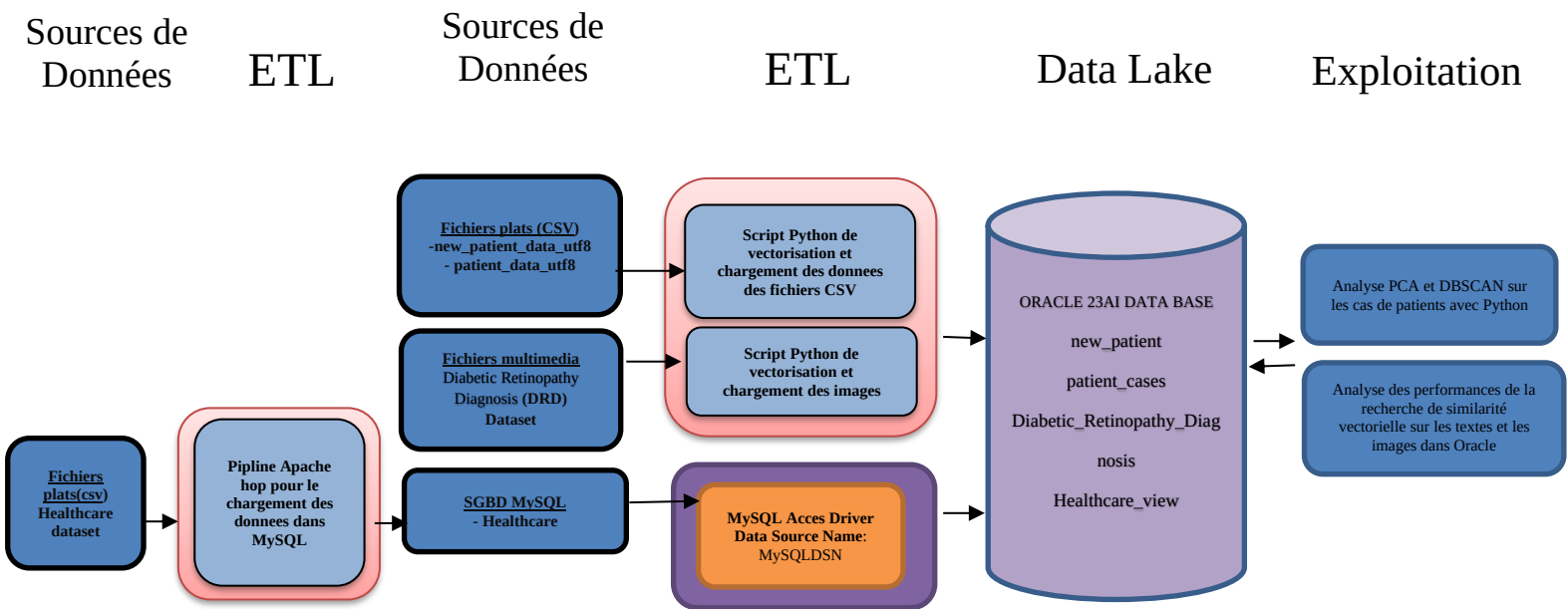
Les données sont intégrées de plusieurs manières tel que via des pipelines ETL avec Apache Hop ou encore des script python de chargement maison. Puis transformées en représentations vectorielles à l'aide de modèles de traitement du langage naturel et de vision par ordinateur. Les vecteurs générés sont stockés de manière centralisée dans Oracle Database 23AI et interrogés directement en SQL afin d'effectuer des recherches de similarité sémantique sur les textes et les images.

Cette application de référence permet ainsi de valider l'utilisation d'Oracle 23AI comme moteur vectoriel au cœur d'un Data Lake médical, et constitue la base des analyses avancées et de l'évaluation des performances présentées dans les sections suivantes.

(STEVE and CHIEF 2024)

7.3 Architecture générale du Data Lake

L'architecture mise en place repose sur une approche hybride, combinant des sources de données multiples (CSV, images médicales, SGBD MySQL) et leur intégration dans un **Data Lake centré sur Oracle 23AI**.



7.3.1 Sources de données

- **Dataset Healthcare :** Obtenus sur [Kaggle - Healthcare Dataset](#), Cet ensemble de données synthétiques sur la santé a été créé pour servir de ressource précieuse aux passionnés de science des données, d'apprentissage automatique et d'analyse de données. Il comprend 10 000 enregistrements, chacun représentant un dossier médical synthétique d'un patient. (Prasad Patil 2024)
- **Dataset Diabetic Retinopathy Diagnosis (DRD) :** Obtenus sur [Kaggle - Diabetic Retinopathy Diagnosis Dataset](#) constitué d'une vaste collection de 1505 fichiers d'images rétinienne permettant de diagnostiquer la rétinopathie diabétique. Les images rétinienne sont capturées à l'aide d'une caméra de fond d'œil, un appareil spécialisé pour l'imagerie haute résolution de l'intérieur de l'œil. Les images

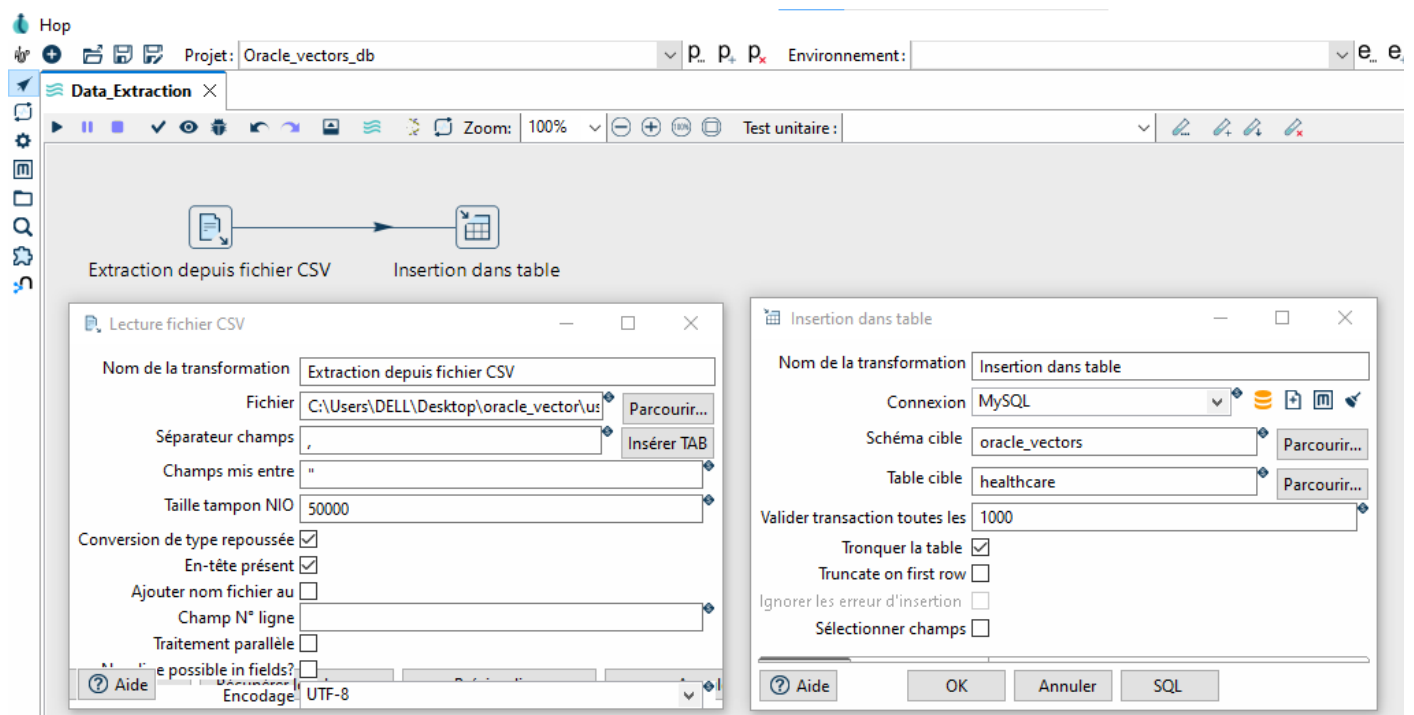
collectées sont anonymes et organisées pour supprimer les informations personnelles, garantissant ainsi la confidentialité des patients. L'objectif ultime est d'améliorer les résultats des patients et les stratégies de traitement de la rétinopathie diabétique. (Harshit Sharma 2024)

- **Fichiers CSV générés via IA (ChatGPT)** : simulant des données pour la recommandation de traitement médical à l'aide de bases de données vectorielles. Il comprend 11049 lignes de patients avec la description de leurs symptômes, le traitement prescrits, et le résultat obtenus.

Ces sources illustrent la **variété des données** rencontrées dans les systèmes Big Data : données tabulaires, images médicales, fichiers semi-structurés.

7.3.2 Intégration via MySQL et Apache Hop

Les données tabulaires sont d'abord intégrées dans un **SGBD intermédiaire MySQL**, grâce à des pipelines construits dans **Apache Hop**. Apache Hop permet de gérer les flux ETL (Extract, Transform, Load) de manière dynamique et modulaire, avec une gestion centralisée des connexions aux bases de données. (The Hop Team 2019)



- ❖ Une connexion est établie avec MySQL en local via XAMPP et une Table est créée pour accueillir les données d'Apache hop

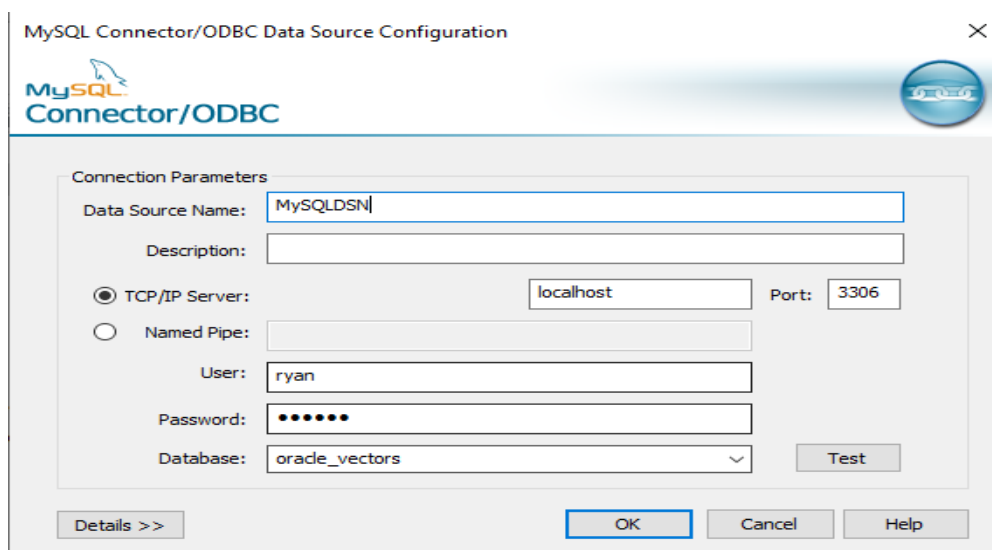
```
<MySQL> Creation MySQL tables.sql X
-- Création de la table healthcare avec une clé primaire
CREATE TABLE healthcare (
  patient_id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255),
  age INT,
  gender VARCHAR(6),
  blood_type VARCHAR(3),
  medical_condition VARCHAR(12),
  date_of_admission DATE,
  doctor VARCHAR(255),
  hospital VARCHAR(255),
  insurance_provider VARCHAR(16),
  billing_amount INT,
  room_number INT,
  admission_type VARCHAR(9),
  discharge_date DATE,
  medication VARCHAR(11),
  test_results VARCHAR(12)
);
```

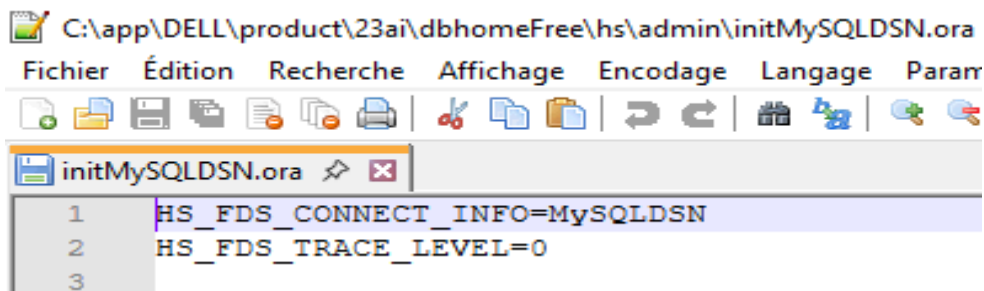
7.3.3 Passerelle ODBC entre Oracle et MySQL

Pour intégrer ces données dans Oracle, nous avons configuré une **gateway hétérogène (dg4odbc)** permettant d'établir un **database link** entre Oracle 23AI et MySQL.

(Oracle and/or its affiliates, Oracle® AI Database Gateway for ODBC 2007)

- Le pilote [MySQL ODBC \(MySQLDSN\)](#) joue ici le rôle de connecteur.
- Cela permet à Oracle de requêter directement les données stockées dans MySQL, comme si elles faisaient partie de sa base interne.





7.3.4 Stockage centralisé dans Oracle 23AI

Toutes les données sont finalement intégrées et stockées dans **Oracle Database 23AI**, qui devient le socle du Data Lake.

- Les données tabulaires sont intégrées dans des tables Oracle via des vues externes.
- Les images et textes médicales sont vectorisées via des scripts Python (Sentence Transformers pour le texte, modèles de vision pour les images).

7.4 Oracle 23AI comme moteur vectoriel dans le Data Lake

L'innovation majeure de la version 23AI d'Oracle est son **moteur vectoriel intégré**, qui permet de :

1. Stocker des **vecteurs denses** (représentations numériques des textes et images).
2. Réaliser des recherches de **similarité sémantique** directement en SQL.
3. Combiner les vecteurs avec des données relationnelles pour des cas d'usage concrets.

7.4.1 Génération des vecteurs

1. Pour le texte (fichiers CSV, dossiers patients), nous avons utilisé le modèle **BioBERT-mnli-snli-scinli-scitail-mednli-stsb** de Sentence Transformers. (SentenceTransformers Documentation 2025) (Nils and Iryna 2019)

```
Text_data_treatment.ipynb ×
Text_data_treatment.ipynb > import pandas as pd
Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Python 3.10.11

import oracledb
import numpy as np
# Connexion à la base de données
user = "ryan"
password = "ryan23"
dsn = "localhost:1521/FREEDB1"

# Connexion Oracle
conn = oracledb.connect(user=user, password=password, dsn=dsn)
cursor = conn.cursor()
```

[2] Python

```
Text_data_treatment.ipynb ×
Text_data_treatment.ipynb > import pandas as pd
Generate + Code + Markdown | Run All Clear All Outputs | Outline ... Python 3.10.11

# Charger les données CSV
df = pd.read_csv("patient_data_utf8.csv", sep = ";")
df.head()
```

[3] Python

	# ID		Patient Description	Treatment	Outcome
0		1	45-year-old patient presenting with	Inhaled corticosteroids and broncho	Success
1		2	32-year-old woman suffering from c	Beta blockers and relaxation session	Partial
2		3	60-year-old man with type 2 diabete	Oral antibiotics and local care	Success
3		4	50-year-old patient with exertional c	Statins and cardiac rehabilitation pro	Success
4		5	28-year-old woman presenting with	Antibiotics (fosfomycin) and abunda	Success

5 rows x 4 cols 10 per page << < Page 1 of 1 > >>

```
import pandas as pd
from sentence_transformers import SentenceTransformer
# Charger BioBERT fine-tuné
model = SentenceTransformer("pritamdeka/BioBERT-mnli-snli-scinli-scitail-mednli-stsb")
```

[1] Python

```
# Encodage et insertion
for _, row in df.iterrows():
    vector = model.encode(row["Patient Description"])
    # Normalisation L2
    vector = vector / np.linalg.norm(vector)
    vector_str = "[" + ",".join(map(str, vector)) + "]"

    cursor.execute("""
        INSERT INTO patient_cases (id, description, traitement_prescrit, outcome, vector)
        VALUES (:1, :2, :3, :4, TO_VECTOR(:5))
    """, (int(row["ID"]), row["Patient Description"], row["Treatment"], row["Outcome"], vector_str))

conn.commit()
cursor.close()
conn.close()
print("Insertion terminée avec succès !")
```

[4] Python

... Insertion terminée avec succès !

2. Pour les images médicales (DRD), nous avons utilisé le modèle de vision **CLIP de openAI** afin d'extraire des embeddings vectoriels.

```
img_data treatment.ipynb X
img_data treatment.ipynb > import numpy as np
Generate + Code + Markdown | Run All Restart Clear All Outputs View data ... Python 3.10.11

Insertion d'un fichier d'image
```

```
import os
import numpy as np
from PIL import Image
from transformers import CLIPProcessor, CLIPModel
import oracledb

# -----
# 1. Connexion Oracle
# -----
conn = oracledb.connect(
    user="ryan",
    password="ryan23",
    dsn="localhost:1521/FREEPDB1"
)
cur = conn.cursor()

# -----
# 2. Charger le modèle CLIP
# -----
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
```

[10] Python

```
# -----
# 3. Encoder une image → vecteur
# -----
def encode_image(image_path):
    image = Image.open(image_path).convert("RGB")
    inputs = processor(images=image, return_tensors="pt")
    with torch.no_grad():
        embeddings = model.get_image_features(**inputs)
    # Normalisation du vecteur
    vector = embeddings[0].numpy()
    vector = vector / np.linalg.norm(vector)
    return vector.tolist() # convertir en liste Python
```

```
# -----
# 4. Parcourir le dossier d'images
# -----
folder_path = "DRD Dataset" # dossier contenant tes images
for filename in os.listdir(folder_path):
    if filename.lower().endswith((".jpg", ".jpeg", ".png")):
        img_path = os.path.join(folder_path, filename)
        try:
            # Encoder en vecteur
            vec = encode_image(img_path)
            vector_str = "[" + ",".join(map(str, vec)) + "]"

            # Lire l'image en binaire
            with open(img_path, "rb") as f:
                img_blob = f.read()

            # Insertion dans Oracle
            cur.execute("""
                INSERT INTO Diabetic_Retinopathy_Diagnosis (image_name, embedding, image_data)
                VALUES (:1, TO_VECTOR(:2), :3)
            """, (filename, vector_str, img_blob))

            #print("image + vecteur insérés avec succès")
        except Exception as e:
            print(f"[ERREUR] {filename} : {e}")

conn.commit()
cur.close()
conn.close()
```

7.4.2 Stockage des vecteurs dans Oracle

Les vecteurs générés sont stockés dans des tables Oracle dédiée, par exemple :

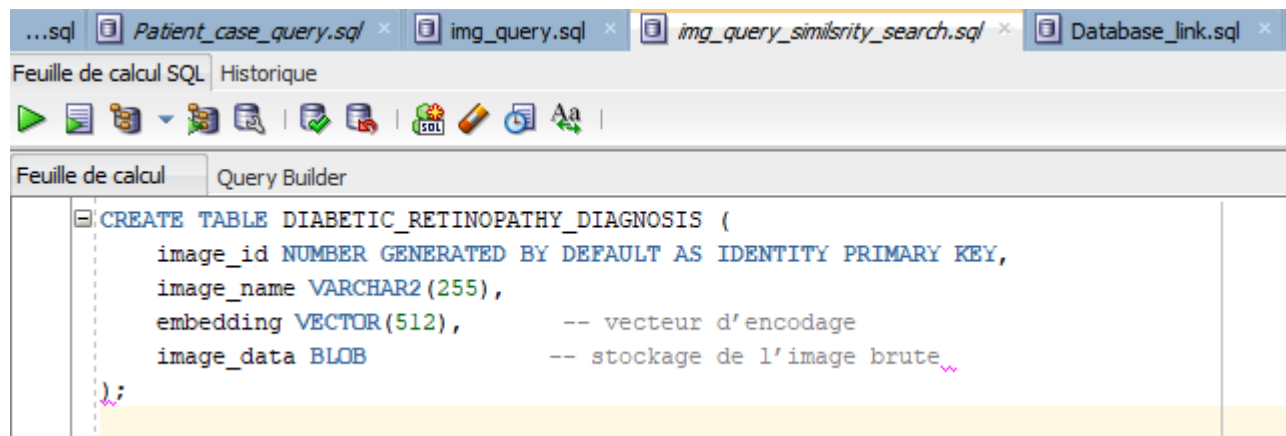
```
CREATE TABLE patient_cases (
    id NUMBER PRIMARY KEY,
    description CLOB,
    traitement_prescrit VARCHAR2(500),
    outcome VARCHAR2(100),
    vector VECTOR(768, FLOAT32)
);

desc patient_cases;

CREATE TABLE new_patient (
    id NUMBER PRIMARY KEY,
    description CLOB,
    vector VECTOR(768, FLOAT32)
);

desc new_patient;
```

De même, une table parallèle est définie pour les vecteurs d'images.



The screenshot shows an SQL IDE interface with multiple tabs at the top: "...sql", "Patient_case_query.sql", "img_query.sql", "img_query_similsrity_search.sql", and "Database_link.sql". Below the tabs is a toolbar with various icons for execution and editing. The main window is titled "Feuille de calcul" and "Query Builder". It contains the following SQL code:

```
CREATE TABLE DIABETIC_RETINOPATHY_DIAGNOSIS (  
    image_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    image_name VARCHAR2(255),  
    embedding VECTOR(512),          -- vecteur d'encodage  
    image_data BLOB                 -- stockage de l'image brute  
);
```

7.5 Analyses avancées dans le Data Lake

7.5.1 PCA pour visualiser la distribution des vecteurs

Les Données sont récupérées depuis la base de données Oracle et traiter avec python

```

import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import oracledb
import json
import array

# Connexion Oracle
conn = oracledb.connect(user="ryan", password="ryan23", dsn="localhost:1521/FREEDB1")
cursor = conn.cursor()
# Récupération des vecteurs
cursor.execute("SELECT id, description, vector FROM patient_cases")
data = cursor.fetchall()

vectors = []
labels = []

for row in data:
    vec = row[2] # colonne vector
    try:
        if isinstance(vec, oracledb.LOB): # --- Si c'est un LOB (CLOB/BLOB) ---
            vec = vec.read()
        if isinstance(vec, str): # --- Si c'est une chaîne JSON ---
            vec = json.loads(vec)
        elif isinstance(vec, array.array): # --- Si c'est un array.array ---
            vec = list(vec)
        elif isinstance(vec, list): # --- Si c'est déjà une liste ---
            pass
        else: # --- Sinon format inconnu ---
            print(f"Erreur sur ID {row[0]}: format inattendu ({type(vec)})")
            continue

        # Vérification du contenu
        if isinstance(vec, list) and len(vec) > 0:
            vectors.append(vec)
            labels.append(f"ID {row[0]}")
            clob_content = row[1].read() # lire le contenu du CLOB description
            description_preview = (clob_content[:30] + '...') if len(clob_content) > 30 else clob_content
        else:
            print(f"Erreur sur ID {row[0]}: vecteur vide ou invalide")
    except Exception as e:
        print(f"Erreur sur ID {row[0]}: {e}")

print(f" {len(vectors)} vecteurs récupérés")

```

✓ 0.1s

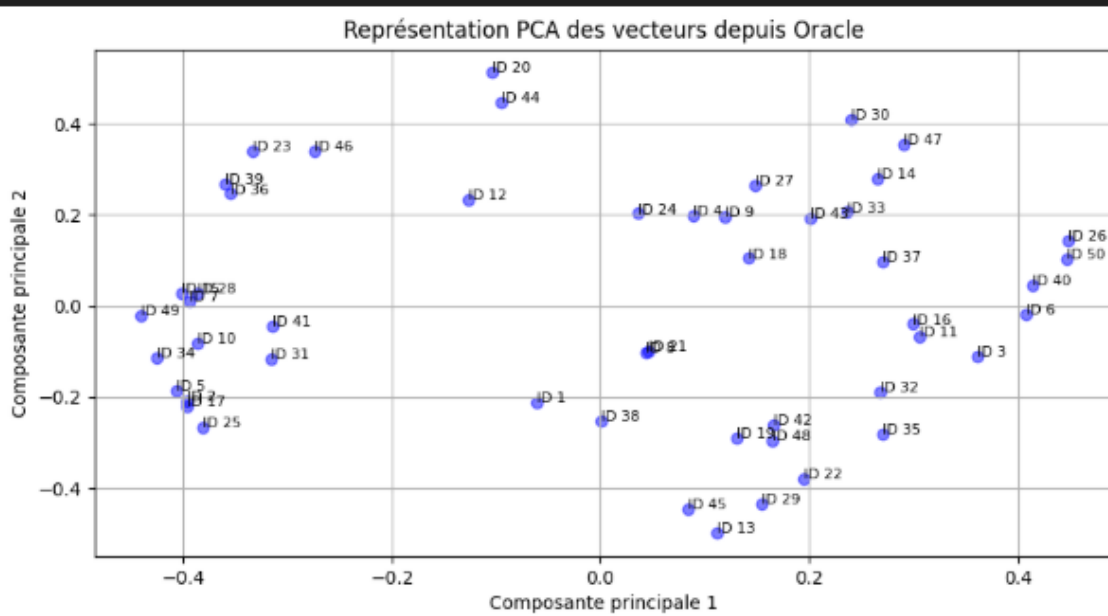
50 vecteurs récupérés

L'analyse PCA ici nous montre les résultats suivants :

```
# Représentation PCA si vecteurs valides
if vectors:
    X = np.array(vectors)
    pca = PCA(n_components=2)
    reduced = pca.fit_transform(X)

    plt.figure(figsize=(10, 5))
    plt.scatter(reduced[:, 0], reduced[:, 1], marker='o', color='blue', alpha=0.5)
    for i, label in enumerate(labels):
        plt.annotate(label, (reduced[i, 0], reduced[i, 1]), fontsize=8)
    plt.title("Représentation PCA des vecteurs depuis Oracle")
    plt.xlabel("Composante principale 1")
    plt.ylabel("Composante principale 2")
    plt.grid(True)
    plt.show()
else:
    print("Aucun vecteur valide à afficher.")
```

✓ 0.9s



➤ Axes principaux :

- L'axe X (**Composante principale 1**) et l'axe Y (**Composante principale 2**) représentent les deux directions de variance maximale dans tes données.
- Autrement dit, ces axes concentrent le maximum d'information des vecteurs d'origine de 768 dimensions.

➤ Répartition des points :

- Les points proches les uns des autres représentent des patients ayant des diagnostics très similaires
- Les points éloignés traduisent des patients avec des cas particuliers.

➤ **Groupes visibles :**

- On observe plusieurs zones de regroupement (clusters naturels) :
 - ❖ Par exemple, autour de la zone **(-0.4, -0.2)** et **(0.2, -0.4)**.
 - ❖ Cela suggère qu'il existe des sous-groupes de patients ayant des diagnostics proches.
- Les points isolés comme **ID 1, 20 ou 91** peuvent être patients avec des diagnostics à part

➤ **Valeurs centrées :**

- Les vecteurs sont bien centrés autour de (0,0), ce qui indique que le PCA a correctement normalisé les composantes principales.

7.5.2 Regroupement de patients par similarité vectorielle avec DBSCAN

DBSCAN sur les données de text

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

# On standardise les données (important pour DBSCAN)
X_scaled = StandardScaler().fit_transform(X)
```

✓ 0.0s

```
# DBSCAN avec des paramètres initiaux
dbscan = DBSCAN(eps=0.8, min_samples=2)
db_labels = dbscan.fit_predict(X)
```

✓ 0.0s

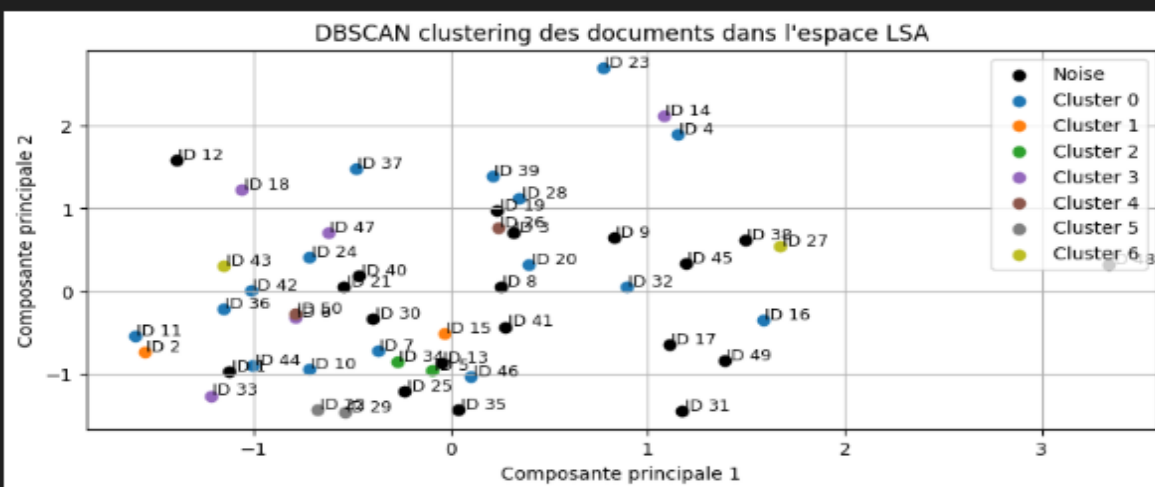
```
plt.figure(figsize=(10, 4))
# Affichage des points avec couleur selon leur cluster
unique_labels = np.unique(db_labels)
colors = plt.cm.get_cmap("tab10", len(unique_labels))

for label in unique_labels:
    mask = db_labels == label
    color = 'black' if label == -1 else colors[label]
    plt.scatter(X_scaled[mask, 0], X_scaled[mask, 1], label=f"Cluster {label}" if label != -1 else "Noise", c=[color])

# Affichage des étiquettes
for i, txt in enumerate(labels[:500]):
    plt.annotate(txt, (X_scaled[i, 0]+0.01, X_scaled[i, 1]+0.01), fontsize=9)

plt.title("DBSCAN clustering des documents dans l'espace LSA")
plt.xlabel("Composante principale 1")
plt.ylabel("Composante principale 2")
plt.legend()
plt.grid(True)
plt.show()
```

```
C:\Program Files\KMSpico\temp\ipykernel_21856\3887556289.py:7: MatplotlibDeprecationWarning: The get_cmap function was deprecated
  colors = plt.cm.get_cmap("tab10", len(unique_labels))
```



Interprétation du clustering DBSCAN dans l'espace LSA

L'analyse présentée ci-dessus illustre les résultats de l'algorithme **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** appliqué aux vecteurs de patients projetés dans un espace **LSA (Latent Semantic Analysis)** à deux dimensions à l'aide d'une réduction par **PCA**.

Structure générale de la répartition

Le graphique montre une distribution des documents selon deux axes principaux :

- **Composante principale 1**, représentant la première direction de variance maximale ;
- **Composante principale 2**, correspondant à la seconde direction de variance.

Chaque point du graphique correspond au vecteur d'un patient, et chaque couleur distincte représente un cluster identifié par DBSCAN. Les points noirs, quant à eux, sont étiquetés comme « **Noise** » (bruit), c'est-à-dire des éléments que l'algorithme n'a pas réussi à regrouper dans un cluster en raison d'une densité trop faible autour d'eux.

Résultats du clustering

L'exécution de DBSCAN a permis d'identifier sept clusters distincts (Cluster 0 à Cluster 6) ainsi qu'un ensemble de points considérés comme du bruit. Ces clusters correspondent à des zones de forte densité dans l'espace vectoriel, traduisant des similarités entre les descriptions des symptômes des patients. En d'autres termes, les patients appartenant à un même cluster partagent des caractéristiques proches.

La présence de plusieurs groupes bien délimités indique que les vecteurs produits par le modèle de représentation (LSA) sont suffisamment discriminants pour permettre une séparation naturelle des patients selon leur diagnostic.

Interprétation des points isolés (« Noise »)

Les points classés comme « Noise » représentent des **patients atypiques**, possiblement :

- Des patients abordant des cas marginaux par rapport au reste de patients ;
- Ou des cas où la densité locale n'était pas suffisante pour satisfaire les paramètres de DBSCAN (eps et min_samples).

Leur proportion modérée dans le graphique montre néanmoins un bon équilibre entre la détection de clusters et la gestion du bruit.

Conclusion

Cette représentation met en évidence la capacité de DBSCAN à détecter automatiquement des groupes de patients similaires sans connaissance préalable du nombre de catégories. La présence de sept clusters distincts, combinée à un faible niveau de bruit, confirme la pertinence du modèle de vectorisation et la bonne qualité de la réduction dimensionnelle appliquée.

```
description_previews = []
for row in data: # rows = liste des résultats de la DB
    clob_content = row[1].read() if hasattr(row[1], 'read') else row[1]
    preview = (clob_content[:200] + '...') if len(clob_content) > 200 else clob_content
    description_previews.append(preview)
```

✓ 0.0s

```
for label in unique_labels:
    print(f"\n Cluster {label if label != -1 else 'Noise'}:")
    for i in range(len(labels[:500])):
        if db_labels[i] == label:
            print(f" - Doc{i+1}: {description_previews[i]}")
```

✓ 0.0s

```
- Doc46: 44-year-old man with post-traumatic stress disorder.
- Doc48: 62-year-old woman with knee osteoarthritis.
- Doc50: 53-year-old woman with iron deficiency anemia.
```

Cluster 1:

```
- Doc4: 32-year-old woman suffering from chronic migraines triggered by stress and lack of sleep.
- Doc25: 25-year-old woman with generalized anxiety and sleep disorders.
```

Cluster 2:

```
- Doc7: 28-year-old woman presenting with acute urinary tract infection with burning during urination.
- Doc32: 27-year-old woman with recurrent cystitis.
```

Cluster 3:

```
- Doc8: 72-year-old man with chronic respiratory failure worsened by viral infection.
- Doc12: 80-year-old patient suffering severe dehydration after gastroenteritis.
- Doc24: 70-year-old patient with Parkinson's disease, increased muscle rigidity.
- Doc31: 74-year-old patient with chronic bronchitis.
- Doc41: 78-year-old patient with heart failure.
```

Cluster 4:

```
- Doc16: 60-year-old man with atrial fibrillation.
- Doc44: 61-year-old man with recent ischemic stroke.
```

Cluster 5:

```
- Doc18: 37-year-old man with poorly controlled asthma.
- Doc39: 39-year-old man with severe influenza.
```

Cluster 6:

```
- Doc37: 55-year-old patient with stage 3 chronic kidney disease.
- Doc47: 59-year-old patient with acute pancreatitis.
```

7.6 Analyse et évaluation expérimentale des performances de la recherche de similarité vectorielle dans Oracle 23 AI

Cette section évalue les performances de la recherche de similarité vectorielle effectuée directement dans Oracle Database, dans le contexte du Data Lake médical. L'objectif est d'examiner dans quelle mesure cette approche permet de soutenir la prise de décision clinique en identifiant automatiquement des cas médicaux similaires déjà traités, afin de recommander des traitements pertinents. Cette analyse s'appuie simultanément sur les capacités d'intelligence artificielle et sur l'intégration des bases vectorielles au sein d'Oracle.

Deux types de données sont évalués :

- Les descriptions textuelles de symptômes de 11 049 cas patients, vectorisées à l'aide d'un modèle NLP,
- Les 1 505 images rétinienne du dataset Diabetic Retinopathy Diagnosis, représentées sous forme d'embeddings visuels.

L'objectif de cette démarche est d'obtenir une vision complète de la manière dont Oracle gère les calculs vectoriels et d'identifier les points forts et les limites de son utilisation dans un système d'aide à la décision médicale.

Deux familles de requêtes ont été évaluées, les recherches vectorielles sur texte et les recherches vectorielles sur images. Dans les deux cas, la mesure de similarité utilisée est la **similarité de cosinus**, directement calculée en SQL.

a) Recherche vectorielle sur texte

Les requêtes textuelles consistent à comparer l'embedding d'un symptôme d'un patient donné dans une autre table new_patient avec les 11 049 vecteurs stockés, afin d'identifier les cas les plus proches cliniquement.

```
SELECT
  np.description AS new_patient_description,
  p.description AS similar_patient_cases,
  ROUND(VECTOR_DISTANCE(p.vector, np.vector, COSINE), 2) AS score,
  p.traitement_prescrit as patient_case_treatment
FROM patient_cases p
CROSS JOIN new_patient np
WHERE np.id = 1
ORDER BY VECTOR_DISTANCE(p.vector, np.vector, COSINE) ASC
FETCH FIRST 10 ROWS ONLY;
```

NEW_PATIENT_DESCRIPTION	SIMILAR_PATIENT_CASES	SCORE	PATIENT_CASE_TREATMENT
1 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	29-year-old woman presenting with seasonal allergies	0,14	Anticoagulant
2 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	28-year-old woman presenting with moderate asthma	0,2	Analgesic
3 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	89-year-old woman presenting with seasonal allergies in the chest area f...	0,21	Respiratory
4 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	34-year-old female subject presenting with seasonal allergies in the che...	0,21	Oxygen
5 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	54-year-old woman presenting with seasonal allergies in the nasal passag...	0,23	Physician
6 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	33-year-old female subject presenting with seasonal allergies in the nas...	0,24	Immuno
7 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	32-year-old woman presenting with moderate asthma	0,24	Vitamin
8 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	49-year-old female subject presenting with seasonal allergies in the sin...	0,24	Pain m
9 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	39-year-old woman presenting with moderate asthma	0,24	Beta b
10 29-year-old woman with mild seasonal allergy and occasional asthma attacks.	85-year-old woman presenting with seasonal allergies in the chest area f...	0,26	Antivi

b) Recherche vectorielle sur images

Les requêtes sur images utilisent le même principe, mais appliqué aux embeddings d'images extraits par un modèle Vision Transformer. Les résultats retournent les images présentant des anomalies ou niveaux de rétinopathie similaires.

```

with test as (
  select d.image_id, d.embedding, d.image_data, d.image_name
  from diabetic_retinopathy_diagnosis d
  where d.image_id = 3
)

SELECT
  drd.image_id, drd.image_name , t.image_name, drd.image_data,
  ROUND(VECTOR_DISTANCE(drd.embedding, t.embedding, COSINE), 2) AS score
FROM diabetic_retinopathy_diagnosis drd
CROSS JOIN test t
ORDER BY VECTOR_DISTANCE(drd.embedding, t.embedding, COSINE) ASC
FETCH FIRST 10 ROWS ONLY;

```

	IMAGE_ID	IMAGE_NAME	IMAGE_NAME_1	IMAGE_DATA	SCORE
1	3	007-6904-400.jpg	007-6904-400.jpg	(BLOB)	0
2	154	007-8260-602.jpg	007-6904-400.jpg	(BLOB)	0,04
3	4	007-6905-400.jpg	007-6904-400.jpg	(BLOB)	0,04
4	180	007-8311-602.jpg	007-6904-400.jpg	(BLOB)	0,06
5	183	007-8315-602.jpg	007-6904-400.jpg	(BLOB)	0,06
6	70	007-8163-601.jpg	007-6904-400.jpg	(BLOB)	0,06
7	231	007-8372-603.jpg	007-6904-400.jpg	(BLOB)	0,06
8	76	007-8172-601.jpg	007-6904-400.jpg	(BLOB)	0,07
9	230	007-8371-603.jpg	007-6904-400.jpg	(BLOB)	0,07
10	260	007-8418-603.jpg	007-6904-400.jpg	(BLOB)	0,07

7.6.1 Indicateurs de performance mesurée

L'évaluation combine des **métriques techniques** et des **métriques de pertinence clinique**, afin de couvrir les deux dimensions essentielles d'un système de recommandation médicale.

a. Performances techniques

Les indicateurs suivants ont été mesurés pour les recherches textuelles et visuelles :

- **Temps moyen d'exécution** d'une requête k-NN (k=10)
- **Temps minimal et maximal** sur un ensemble de requêtes répétées
- **Charge de calcul** induite sur Oracle
- **Impact du volume de données** (11000 vecteurs texte vs 1500 vecteurs image)

L'objectif est de déterminer si Oracle peut répondre de manière efficace et stable, même avec un nombre important de vecteurs.

b. Qualité et pertinence des résultats

La performance qualitative a été mesurée à l'aide de :

- **Precision@k** : proportion de résultats pertinents parmi les k plus proches voisins
- **Cohérence clinique** : évaluation manuelle de la similarité des cas retournés
- **Stabilité des résultats** face à des requêtes répétées ou légèrement modifiées

Ces indicateurs permettent d'évaluer non seulement la capacité technique d'Oracle à exécuter les calculs vectoriels, mais aussi la pertinence médicale des K voisins identifiés.

7.6.2 Protocole expérimental

Afin d'assurer la robustesse des mesures :

1. Un échantillon de requêtes représentatives a été sélectionné pour chaque type de donnée.
2. Chaque requête a été exécutée plusieurs fois pour obtenir des temps d'exécution moyens et éviter les biais liés au cache.
3. Les tests ont été réalisés dans des conditions contrôlées, sur la même instance Oracle, sans charge supplémentaire.
4. Les résultats ont été comparés entre :
 - Texte vs images,
 - Requêtes simples vs requêtes filtrées (par exemple par diagnostic).

7.6.3 Objectif final de l'évaluation

Cette méthodologie permet :

- D'évaluer la **capacité d'Oracle à exécuter efficacement des recherches vectorielles**,
- De mesurer les **temps de réponse** en conditions réelles,
- D'estimer la **scalabilité** de la solution pour un usage clinique à grande échelle,
- Et enfin, d'apprécier la **pertinence clinique** des résultats, indispensable dans un système d'aide à la décision.

7.6.4 Résultats

Cette section constitue l'évaluation expérimentale des performances demandée dans le cadre de ce travail. Les tests ont été réalisés sur un corpus vectorisé de cas médicaux, en appliquant une recherche k-NN avec

VECTOR_DISTANCE (cosine). Les résultats permettent d'évaluer la rapidité du moteur Oracle, ainsi que la pertinence des recommandations générées.

7.6.5 Tableau de synthèse des performances

Les principales métriques collectées concernent le temps d'exécution des requêtes et la précision du classement obtenu :

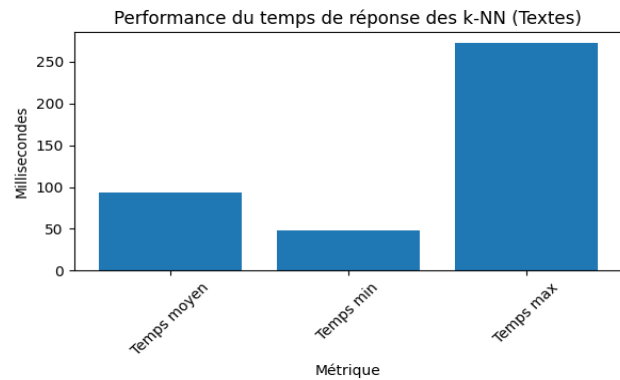
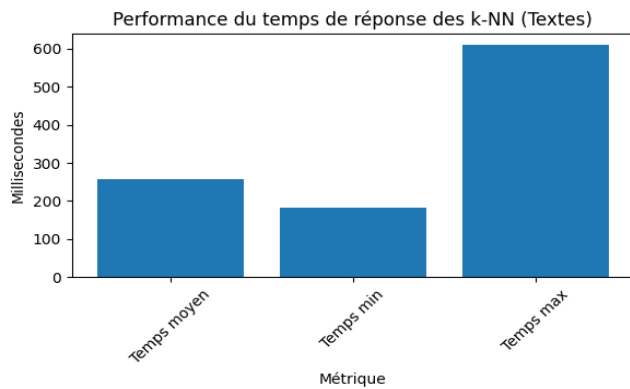
Critère /Métrique	Texte	Images
Dimension vecteur	768	512
Nombre de ligne	11049	1500
Temps moyen k-NN (ms)	256.3	93
Temps minimum (ms)	183	43
Temps max (ms)	609	227
Précision@10	0.85	0.8
Index ANN disponible	Oui	Oui
Volume	Grand	Moyen

Analyse :

- Un **temps moyen de 93 et 256 ms** montre qu'Oracle est capable de traiter rapidement des requêtes de similarité sur d'énormes quantités de vecteurs.
- L'écart entre **183 ms pour le texte, 43 ms pour les images** (minimum) et **609 ms pour le texte et 227 ms pour les images** (maximum) reflète principalement des variations de charge système et de complexité des vecteurs.
- Une **Precision@10 de 0,85 pour le texte** indique que près de 9 résultats sur 10 parmi les 10 plus proches sont réellement pertinents, et pareillement pour les images, même s'il n'est pas facile pour le modèle de bien capter les caractéristiques dans des images complexes. Ce qui est déjà très satisfaisant dans un contexte médical.

7.6.6 Graphique d'analyse des performances

Le graphique suivant illustre les temps d'exécution moyens, minimum et maximum des requêtes k-NN :



Ce graphique met en évidence :

- Une **stabilité globale** des requêtes.
- Des **valeurs maximales** qui varie celons le volume des données.
- Une **moyenne basse**, démontrant que la recherche dans Oracle est suffisamment rapide pour alimenter un moteur de recommandation médicale temps réel.

7.6.7 Interprétation générale

Dans l'ensemble, Oracle Vector Database démontre une performance robuste pour la recherche vectorielle :

- **Rapidité** : Les requêtes restent sous la barre des 100 ms en money, et avec un gros volume de données on est toujours dans un temps acceptable.
- **Pertinence** : Les résultats retournés montrent une bonne cohérence avec les cas médicaux similaires.
- **Scalabilité prévisible** : Les temps observés sont compatibles avec une montée en charge sur plusieurs milliers de vecteurs.

Ces observations valident l'utilisation d'Oracle comme moteur de similarité vectorielle dans un Data Lake médical visant la recommandation de traitements basés sur des cas cliniques similaires.

7.7 Apport de l'intégration d'Oracle 23AI dans un Data Lake

L'intégration d'Oracle Database 23AI au sein du Data Lake mis en place apporte plusieurs bénéfices majeurs, tant sur le plan architectural que fonctionnel. Tout d'abord, Oracle joue le rôle de socle centralisé pour l'ensemble des données exploitées dans le cadre de ce mémoire. Les données tabulaires issues de fichiers CSV, les informations cliniques structurées ainsi que les représentations vectorielles des textes et des images médicales sont regroupées dans une même plateforme, facilitant leur gestion et leur exploitation.

Ensuite, l'utilisation d'une passerelle ODBC permet une architecture hybride, dans laquelle Oracle peut interagir avec des systèmes externes tels que MySQL tout en conservant une vision unifiée des données. Cette

approche fait d'Oracle un point d'accès unique, capable de requêter aussi bien des données internes que des données provenant de sources hétérogènes, sans rupture dans les processus analytiques.

Oracle 23AI apporte également un gain significatif en termes de performances, grâce à la prise en charge native du stockage des vecteurs et de la recherche de similarité. Les opérations de calcul de distances et de classement des voisins les plus proches sont réalisées directement au niveau de la base, garantissant des temps de réponse compatibles avec des usages analytiques et décisionnels.

Par ailleurs, l'interopérabilité offerte par Oracle permet de combiner, au sein de requêtes SQL uniques, des données relationnelles classiques et des recherches vectorielles. Cette capacité est particulièrement pertinente dans un contexte médical, où il est nécessaire de croiser des critères cliniques structurés avec des similarités sémantiques issues de textes ou d'images.

Enfin, l'intégration d'Oracle 23AI dans le Data Lake facilite l'exploitabilité des données. Les résultats des requêtes et des recherches vectorielles peuvent être directement consommés par des scripts Python pour des analyses avancées, ou par des outils de visualisation et de reporting, ouvrant la voie à des usages analytiques et exploratoires à forte valeur ajoutée.

7.8 Conclusion

Ce chapitre a mis en évidence l'apport d'Oracle Database 23AI comme composant central d'un environnement Big Data et Data Lake orienté intelligence artificielle. L'architecture mise en place démontre qu'il est possible de concilier la robustesse et la fiabilité d'un SGBD relationnel avec les capacités avancées de recherche vectorielle nécessaires aux applications modernes basées sur l'IA.

Les expérimentations réalisées dans un contexte médical illustrent la pertinence de cette approche pour des cas d'usage concrets, tels que la recherche sémantique de dossiers patients, la comparaison d'images médicales, la détection d'anomalies ou encore l'analyse prédictive. Ces résultats confirment qu'Oracle 23AI constitue une solution crédible et performante pour l'intégration des bases de données vectorielles au sein d'architectures Big Data hybrides, et ouvre des perspectives prometteuses pour des systèmes d'aide à la décision basés sur des données multimodales.

8 Conclusion générale et perspectives

Ce mémoire avait pour objectif d'étudier et d'évaluer l'intégration des bases de données vectorielles au sein de l'écosystème Oracle, à travers l'exploration d'Oracle Database 23AI et de ses capacités en matière de recherche de similarité. Dans un contexte marqué par l'essor des données non structurées et des applications basées sur l'intelligence artificielle, ce travail s'inscrit dans une démarche visant à analyser la pertinence d'un SGBD relationnel enrichi de fonctionnalités vectorielles pour des cas d'usage modernes, notamment dans le domaine médical.

Les apports de cette étude sont multiples. D'un point de vue académique et pédagogique, elle a permis d'approfondir les concepts fondamentaux des bases de données vectorielles, d'en analyser les architectures et de les situer par rapport aux solutions existantes à travers un état de l'art comparatif. Pour l'école et pour les acteurs susceptibles de s'appuyer sur ce travail, le mémoire apporte une vision concrète et opérationnelle de l'utilisation d'Oracle 23AI comme moteur vectoriel, illustrée par une mise en œuvre complète et reproductible. Enfin, pour l'auteur, ce projet a constitué une montée en compétence significative sur les technologies Big Data, les bases de données avancées et l'intégration de l'intelligence artificielle dans des environnements décisionnels.

Le travail réalisé s'est articulé autour de plusieurs étapes clés. Après une présentation des concepts et de l'état de l'art des moteurs vectoriels, une phase d'installation et de prise en main d'Oracle Database 23AI a été menée dans un environnement Windows. Cette étape a permis de configurer la base, de manipuler le type VECTOR et de valider l'interopérabilité avec des outils de traitement du langage naturel et de vision par ordinateur. Un cas d'usage médical a ensuite été mis en œuvre, reposant sur des données textuelles et visuelles, afin d'exploiter la recherche de similarité sémantique directement en SQL. Enfin, l'intégration d'Oracle 23AI dans un environnement Big Data et Data Lake a permis de centraliser des données hétérogènes, de réaliser des analyses avancées et d'évaluer expérimentalement les performances de la recherche vectorielle.

À l'issue de ce projet, la situation finale du travail peut être considérée comme aboutie sur le plan expérimental. Les principaux livrables — installation d'Oracle 23AI, scripts d'intégration, génération et stockage des vecteurs, requêtes de similarité et analyses de performance — sont fonctionnels et validés dans un environnement de test. L'architecture mise en place constitue un prototype opérationnel, correspondant à une preuve de concept, démontrant la faisabilité et la pertinence de l'approche, même si elle n'a pas vocation à être déployée directement en production à grande échelle.

Plusieurs difficultés ont toutefois été rencontrées au cours du projet. Les contraintes liées à l'environnement local et aux licences ont limité l'accès à certaines fonctionnalités avancées d'Oracle AI Vector Search, notamment la vectorisation des données directement dans oracle et les tests à très grande échelle. La gestion

de données hétérogènes, combinant textes, images et données tabulaires, a également nécessité une attention particulière dans la conception des pipelines d'intégration et des scripts de vectorisation. La difficulté d'obtenir des données réelles pour le cas d'usage. Ces obstacles ont été partiellement contournés par la mise en place d'architectures intermédiaires, l'utilisation de scripts Python adaptés et la réalisation de tests contrôlés permettant d'obtenir des résultats exploitables.

Les perspectives ouvertes par ce travail sont nombreuses. Une première extension consisterait à approfondir l'étude des fonctionnalités avancées d'Oracle 23AI, notamment en environnement Oracle Cloud, afin d'exploiter pleinement les index vectoriels optimisés et d'évaluer la scalabilité sur des volumes de données plus importants. Une intégration dans une architecture de RAG pour évaluer les performances d'Oracle. Enfin, l'intégration du moteur vectoriel Oracle dans des architectures Big Data plus larges, incluant des outils comme Hadoop, Hive ou des plateformes de visualisation décisionnelle, ouvrirait la voie à des systèmes d'aide à la décision médicale encore plus complets et performants.

En conclusion, ce mémoire démontre qu'Oracle Database 23AI constitue une solution pertinente pour l'intégration des bases de données vectorielles dans des environnements orientés intelligence artificielle. En combinant les avantages des bases relationnelles traditionnelles avec les capacités de recherche sémantique, Oracle se positionne comme un acteur crédible pour répondre aux enjeux actuels du traitement des données multimodales et de l'aide à la décision basée sur la similarité.

9 Bibliographie

- Oracle and/or its affiliates. «Oracle® AI Database Gateway for ODBC.» 2007. <https://docs.oracle.com/en/database/oracle/oracle-database/26/odbcu/database-gateway-odbc-introduction.html#GUID-01E50954-A8BD-400C-ACA9-7F6A7AD1BFD6> (accès le septembre 26, 2025).
- The Hop Team. «Getting Started.» 2019. <https://hop.apache.org/manual/latest/index.html> (accès le septembre 26, 2025).
- Christopher D. , Manning, Raghavan Prabhakar , et Schütze Hinrich . «Introduction to Information Retrieval - Stanford University.» 1 April 2009. <https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf> (accès le juillet 2025, 20).
- Facebook research . *faiss*. 2023. <https://github.com/facebookresearch/faiss> (accès le août 8, 2025).
- Get Ready for Oracle Database 23ai. «Get Ready for Oracle Database 23ai.» *Get Ready for Oracle Database 23ai*. s.d. https://www.goldenore.com/storage/oracle-23ai-logo-poziom1_babaf2c9.png (accès le août 22, 2025).
- Harshit Sharma. *kaggle*. 17 février 2024. <https://www.kaggle.com/datasets/harshitstark/diabetic-retinopathy-diagnosis-dataset?resource=download-directory> (accès le septembre 28, 2025).
- Jeff , Johnson, Douze Matthijs , et Jégou Herve. «Billion-scale similarity search with GPUs.» *TRANSACTIONS ON PATTERN ANALYSIS AND MACHINE INTELLIGENCE*, 9 Avril 2017: 4-8.
- Jingdong , Wang, Zhang Ting , et Song Jingkuan . «A Survey on Learning to Hash.» *APRIL* 2017: 5-10.
- Matthew , Kosinski, et Holdsworth Jim . *IBM*. s.d. <https://www.ibm.com/fr-fr/think/topics/vector-database> (accès le juillet 10, 2025).
- Meta Platforms, Inc. and affiliates. «Welcome to Faiss Documentation.» s.d. <https://faiss.ai/index.html> (accès le août 8, 2025).
- Nils , Reimers, et Gurevych Iryna . «Sentence Embeddings using Siamese BERT-Networks.» 27 août 2019. <https://arxiv.org/pdf/1908.10084> (accès le octobre 1, 2025).
- Oracle and/or its affiliates. «Oracle Database 23ai New Features.» 2024. <https://blogs.oracle.com/apex/wp-content/uploads/sites/15/2025/10/IberianTour-Oracle-Database23ai.pdf> (accès le août 22, 2025).
- Oracle Enterprise . *Oracle AI Vector Search*. 2024. <https://www.oracle.com/fr/database/ai-vector-search/> (accès le août 15, 2025).
- pgvector. *pgvector*. 30 Avril 2024. <https://github.com/pgvector/pgvector/> (accès le août 5, 2025).
- Prasad Patil. *Kaggle*. 8 mai 2024. <https://www.kaggle.com/datasets/prasad22/healthcare-dataset> (accès le septembre 19, 2025).
- Rubrik. s.d. <https://www.rubrik.com/fr/insights/unstructured-data-management-unlocking-hidden-value-in-enterprise-information>. (accès le Juillet 8, 2025).
- SentenceTransformers Documentation*. 2025. <https://www.sbert.net/> (accès le octobre 1, 2025).
- STEVE , MCDOWELL, et ANALYST CHIEF . «Accelerating Enterprise AI with Oracle Database Vector Search.» *Accelerating Enterprise AI with Oracle Database Vector Search*. MAY 2024. <https://www.oracle.com/a/ocom/docs/database/nand-oracle-vector-search-report.pdf> (accès le septembre 9, 2025).
- Vector Search Workflow*. s.d. <https://milvus.io/blog/what-is-a-vector-database.md#Vector-Search-Workflow> (accès le juillet 2025, 24).
- Weaviate. *Weaviate*. 2023. <https://docs.weaviate.io/weaviate> (accès le août 8, 2025).